PI_Curs_05

5.0

Bine ați venit la un capitol crucial care a redefinit modul în care rețelele neuronale procesează datele: **5. Mecanisme de Atenție (Attention Mechanisms)**. Inspirată de biologia umană, această lecție ne arată cum să acordăm importanță selectivă informației de intrare.

Ghid de Studiu Complet: 5. Mecanisme de Atenție

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5 introduce Mecanismele de Atenție, un concept care își trage inspirația direct din **sistemul vizual uman**. Nevoia de a folosi atenția provine din faptul că nervul optic primește un *input* senzorial masiv care depășește capacitatea de procesare a creierului. Prin urmare, oamenii au dezvoltat capacitatea de a direcționa **focalizarea și concentrarea** conștiinței către obiectele de interes (precum prăzi sau prădători). În *deep learning*, acest lucru se traduce prin abilitatea modelului de a alege o **mică fracțiune de informație** relevantă dintr-un set mare de date și de a o folosi pentru a genera o ieșire. Baza matematică a atenției este dată de **Attention Pooling**, care folosește o **Query ($\mathbf{q}$)** pentru a calcula scoruri de relevanță față de setul de **Keys ($\mathbf{k}_i$)** și apoi ponderează vectorii **Value ($\mathbf{v}_i$)**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Filtrarea Biologică:** Atenția este un mecanism de **filtrare și focalizare** inspirat de evoluție, care permite procesarea eficientă a *input*-ului senzorial masiv.
2. **Attention Pooling (Suma Ponderată):** Output-ul atenției este întotdeauna o **sumă ponderată** a vectorilor *value* ($\mathbf{v}_i$), unde ponderile sunt date de funcția *softmax*.
3. **Modelul Q-K-V:** Atenția funcționează pe principiul că un *query* ($\mathbf{q}$) este comparat cu o colecție de *keys* ($\mathbf{k}$) pentru a obține scoruri de aliniere ($a(\mathbf{q}, \mathbf{k})$).
4. **Tipuri de Aliniere:** Scorurile de aliniere pot fi calculate prin funcții complexe învățate (precum **Atenția Aditivă**) sau prin operații simple (precum **Atenția Produsului Scalar Scalat**).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5. Mecanisme de Atenție (Context)

- Nervul optic al sistemului nostru vizual primește *input* senzorial masiv, depășind capacitatea creierului de a procesa integral.
- Din fericire, nu toți stimulii sunt egali.
- Focalizarea și concentrarea conștiinței ne permit să direcționăm atenția către obiecte de interes (prăzi, prădători) în mediul vizual complex.
- Abilitatea de a acorda atenție doar unei mici fracțiuni din informație are o semnificație evoluționistă.

Formulele de Bază (Attention Pooling)

- Atenția combină informațiile printr-o sumă ponderată a vectorilor *value* ($\mathbf{v}_i$).
- Output-ul atenției este o funcție a *query*-ului $\mathbf{q}$ și a perechilor *key-value* $(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)$.
- Formula de **Attention Pooling**:

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \in \mathbb{R}$$

.

  - $\mathbf{q} \in \mathbb{R}^Q$ (dimensiunea *query*-ului).
  - $\mathbf{k}_i \in \mathbb{R}^K$ (dimensiunea *key*-ului), $\mathbf{v}_i \in \mathbb{R}$ (dimensiunea *value*-ului).
  - m este numărul de perechi *key-value*.

Calculul Ponderilor de Atenție (α - Softmax)

- Ponderile de atenție $\alpha(\mathbf{q}, \mathbf{k}_i)$ sunt calculate folosind funcția **softmax** aplicată scorurilor de aliniere $a(\mathbf{q}, \mathbf{k}_i)$:

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \mathrm{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{=1}^{m} \exp(a(\mathbf{q}, \mathbf{k}))} \in \mathbb{R}$$

.

Tipuri de Scorul de Aliniere ($a(\mathbf{q}, \mathbf{k})$)

1. **Atenția Aditivă (*Additive Attention*):**
   - Folosește o transformare neliniară (tanh) și parametri învățați.
   - Formula: $a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}) \in \mathbb{R}$.
   - Parametrii sunt: $\mathbf{W}_q \in \mathbb{R}^{h \times q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times k}$, și $\mathbf{w}_v \in \mathbb{R}^h$.
   - h este dimensiunea stratului ascuns.
2. **Atenția Produsului Scalar Scalat (*Scaled Dot-Product Attention*):**
   - O metodă mai simplă, unde $q = k = d$.
   - Formula: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k} / \sqrt{d} \in \mathbb{R}$.

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Ponderarea (Softmax)

- **Explicație Simplificată (Alegerea Feliei de Tort):** Softmax se asigură că ponderile de atenție acționează ca o distribuție de probabilitate. Dacă ai trei *value*-uri ($\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3$), softmax îți spune exact cât la sută din **atenția totală** (100%) să aloci fiecărui *value*. Dacă $a(\mathbf{q}, \mathbf{k}_1)$ este mult mai mare decât celelalte, $\alpha(\mathbf{q}, \mathbf{k}_1)$ va fi aproape de 1, indicând că modelul ar trebui să se uite aproape exclusiv la $\mathbf{v}_1$.

Concept Dificil: Scalarea în Produsul Scalar Scalat

- **Context:** De ce se împarte produsul scalar la $\sqrt{d}$?
- *Asta se întâmplă deoarece:* Pe măsură ce dimensiunea *key*-urilor (d) crește, produsul scalar $\mathbf{q}^\top \mathbf{k}$ tinde să devină foarte mare (sau foarte mic/negativ) ca mărime absolută. Dacă intrările în *softmax* sunt prea mari, gradientul *softmax* devine foarte mic. **Scalarea cu** $\sqrt{d}$ ajută la menținerea varianței produselor scalare sub control, stabilizând astfel *softmax*-ul și antrenamentul.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Context Dinamic):**
  - **CAUZĂ:** Folosirea RNN-urilor simple în Seq2Seq forța condensarea întregii secvențe de intrare într-un singur vector de context (c) de lungime fixă [Previous conversation].
  - **EFECT:** Mecanismul de atenție înlocuiește vectorul c static cu un **vector de context dinamic** ($f(\mathbf{q}, \ldots)$), calculat la fiecare pas de decodificare. Acest lucru permite decodificatorului să se uite înapoi la elementele relevante ale secvenței sursă, prevenind **"gâtul de sticlă"** informațional.
- **Implicația asupra Performanței:** Abilitatea de a pondera informația la nivelul fiecărui *token* (sau *value*) este motivul pentru care Atenția a transformat performanța în NLP, permițând modelarea dependențelor pe distanțe mult mai mari decât RNN-urile cu porți (GRU/LSTM).

---

5. EXERCITII (Calcul și Analiză bazate pe 05_08_Exercises.pdf)

Sursa **05_08_Exercises.pdf** oferă un model de exercițiu detaliat, demonstrând calculul pentru ambele tipuri de atenție (Aditivă și Produs Scalar Scalat) pe același set de date. Voi genera un exercițiu nou, similar ca structură, dar cu date diferite.

Set de Date (Folosit pentru E.1 și E.2):

- $m = 2$ perechi *key-value*. $k = 2, v = 2$.
- Query: $\mathbf{q} = [2\ 1]$.
- Perechile Key-Value:

$$(\mathbf{k}_1, \mathbf{v}_1) = ([1\ 0], [4\ 0]), \quad (\mathbf{k}_2, \mathbf{v}_2) = ([0\ 1], [0\ 5])$$

Exercițiul E.1: Calculul Produsului Scalar Scalat și Atenția Finală

Utilizați setul de date de mai sus. Calculați output-ul final al *Attention Pooling* folosind formula de **Atenție Produs Scalar Scalat**.

- $d = k = 2$. (Factorul de scalare este $1/\sqrt{2} \approx 0.7$).
- Presupuneri pentru Softmax: $\exp(1.4) \approx 4.05, \exp(0.7) \approx 2.0$.

1. Calculați scorurile de aliniere $a(\mathbf{q}, \mathbf{k}_1)$ și $a(\mathbf{q}, \mathbf{k}_2)$.
2. Calculați ponderile de atenție $\alpha(\mathbf{q}, \mathbf{k}_1)$ și $\alpha(\mathbf{q}, \mathbf{k}_2)$ folosind funcția *softmax* și aproximările date.
3. Calculați output-ul final $f(\mathbf{q}, \mathbf{k}_1, \mathbf{v}_1, \mathbf{k}_2, \mathbf{v}_2)$.

Rezolvare E.1:

1. **Calculul Scorurilor de Aliniere ($a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}/\sqrt{d}$):**
   - $d = 2 \quad 1/\sqrt{d} = 1/\sqrt{2} \approx 0.7$.
   - $$a(\mathbf{q}, \mathbf{k}_1) = ([2 \quad 1]\,[1\ 0]) \cdot 0.7 = (2) \cdot 0.7 = \mathbf{1.4}$$
   - $$a(\mathbf{q}, \mathbf{k}_2) = ([2 \quad 1]\,[0\ 1]) \cdot 0.7 = (1) \cdot 0.7 = \mathbf{0.7}$$
2. **Calculul Ponderilor de Atenție (α):**
   - Denominatorul Softmax: $D = \exp(1.4) + \exp(0.7) \approx 4.05 + 2.0 = 6.05$.
   - $$\alpha_1 = \frac{\exp(1.4)}{D} \approx \frac{4.05}{6.05} \approx \mathbf{0.67}$$
   - $$\alpha_2 = \frac{\exp(0.7)}{D} \approx \frac{2.00}{6.05} \approx \mathbf{0.33}$$
3. **Calculul Output-ului Final ($f(\mathbf{q}, \ldots)$):**

$$f(\mathbf{q}, \ldots) = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2$$

$$f(\mathbf{q}, \ldots) \approx 0.67\,[4\ 0] + 0.33\,[0\ 5]$$

$$f(\mathbf{q}, \ldots) \approx [2.68\ 0] + [0\ 1.65] = [2.68\ 1.65]$$

Exercițiul E.2: Analiza Atenției Aditive (Dimensionalitate și Parametri)

Se definește un model de Atenție Aditivă cu următoarele dimensiuni:

- Dimensiunea *Query* $q = 10$.
- Dimensiunea *Key* $k = 10$.
- Dimensiunea stratului ascuns $h = 20$.

1. Specificați dimensiunea matricilor de ponderi $\mathbf{W}_q$, $\mathbf{W}_k$ și a vectorului $\mathbf{w}_v$.
2. Calculați numărul total de parametri necesari pentru a defini scorul de aliniere al Atenției Aditive.

Rezolvare E.2:

1. **Dimensiunea Parametrilor:**
   - $\mathbf{W}_q \in \mathbb{R}^{h \times q} \quad \mathbf{W}_q \in \mathbb{R}^{20 \times 10}$
   - $\mathbf{W}_k \in \mathbb{R}^{h \times k} \quad \mathbf{W}_k \in \mathbb{R}^{20 \times 10}$
   - $\mathbf{w}_v \in \mathbb{R}^{h} \quad \mathbf{w}_v \in \mathbb{R}^{20}$
2. **Numărul Total de Parametri:**

$$\text{măr}_q = 20 \times 10 = 200$$

$$\text{măr}_k = 20 \times 10 = 200$$

$$\text{măr}_v = 20$$

$$\text{otaparametri} = 200 + 200 + 20 = \mathbf{420}$$

Exercițiul E.3: Rolul Evoluționar al Atenției (Conceptual)

De ce menționează sursa că abilitatea de a acorda atenție doar unei mici fracțiuni de informație are o **semnificație evoluționistă**?

Rezolvare E.3:

Semnificația evoluționistă derivă din **supraîncărcarea cognitivă**. Prin focalizarea atenției, sistemul nervos:

1. **Conservă resurse:** Evită risipa de energie computațională prin încercarea de a procesa integral *input*-ul senzorial masiv.
2. **Permite Răspuns Rapid:** Asigură că informația critică (de exemplu, detectarea rapidă a unui prădător) primește prioritate, permițând indivizilor să reacționeze rapid și eficient în medii complexe.
3. **Supraviețuire și Succes:** Abilitatea de a filtra zgomotul și de a se concentra pe elementele vitale (precum prăzile și prădătorii) este direct legată de capacitatea umană de a supraviețui și de a avea succes.

5.1

Absolut! Lecția **5.1 Attention Cues (Elemente de Atenție)** stabilește fundamentele biologice și conceptuale ale mecanismului de atenție, explicând cum creierul uman filtrează informația și cum este transpus acest concept în *deep learning* prin intermediul modelului Query-Key-Value (QKV).

Ghid de Studiu Complet: 5.1 Elementele de Atenție (Attention Cues)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5.1 explică necesitatea Mecanismelor de Atenție pornind de la un principiu biologic fundamental: **informația în mediul nostru nu este rară, iar creierul nu o poate procesa pe toată**. De exemplu, nervul optic primește informații de ordinul a 10^8 biți pe secundă, depășind capacitatea completă de procesare a creierului. Soluția evoluționistă a fost dezvoltarea capacității de a direcționa atenția către o mică fracțiune de informații de interes, permițând alocarea inteligentă a resurselor pentru supraviețuire, dezvoltare și socializare (precum detectarea prădătorilor sau a prăzilor). Cadrul teoretic care explică modul în care este implementată atenția în sistemul vizual uman se bazează pe două elemente (cues): **nonvolitional cue (saliency)** și **volitional cue (task-dependent control)**.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Contextul Biologic:** Atenția este un mecanism necesar deoarece nu toți *input*-urile senzoriale sunt egale, iar volumul de informație depășește capacitatea de procesare.
2. **Cadrul cu Două Componente:** Atenția este explicată printr-un cadru cu două elemente (cues): **nonvolitional cue** și **volitional cue**.
3. **Elementul Nonvolitional (Key/Saliency):** Este bazat pe **saliency** (proeminența obiectelor din mediu). Acesta este un proces automat și involuntar (ex: o ceașcă roșie într-un mediu alb-negru).
4. **Elementul Volitional (Query):** Este bazat pe **control cognitiv și volitional** și este dependent de sarcină (*task-dependent*), permițând selectarea țintei pe baza unor criterii variabile (ex: dorința de a citi o carte după ce am băut cafeaua).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.1 Attention Cues (Elemente de Atenție)

Necesitatea și Sursa Biologică a Atenției

- Informația din mediul înconjurător nu este rară.

- Când inspectăm o scenă vizuală, nervul optic primește informații la ordinul a 10^8 **biți pe secundă**, depășind cu mult ceea ce creierul nostru poate procesa integral.
- Din fericire, am învățat din experiență (date) că **nu toate *input*-urile senzoriale sunt egale**.
- Capacitatea de a direcționa atenția către o fracțiune de informații de interes a permis creierului să aloce resurse mai inteligent pentru supraviețuire, dezvoltare și socializare (de exemplu, detectarea prădătorilor, prăzilor și partenerilor).

Cadrul de Atenție cu Două Componente (Volitional și Nonvolitional)

- Pentru a explica modul în care atenția este desfășurată în lumea vizuală, a apărut un cadru cu două componente.
- Subiecții direcționează selectiv *spotlight*-ul atenției folosind ambele elemente: **nonvolitional cue** și **volitional cue**.

1. Elementul Nonvolitional (Keys - Saliency)

- Este bazat pe **saliency** (proeminența) obiectelor din mediu.
- Exemplu: Se imaginează cinci obiecte (ziar, hârtie de cercetare, ceașcă de cafea, caiet, carte).
- Dacă toate produsele din hârtie sunt alb-negru, dar ceașca de cafea este **roșie**, ceașca este intrinsec **salientă**.
- Această salience atrage **automat și involuntar** atenția.
- În acest caz, fovea (centrul maculei, unde acuitatea vizuală este cea mai mare) este adusă pe ceașca de cafea (Figura 1).

2. Elementul Volitional (Queries - Task-Dependent)

- Acesta este diferit de cazul *saliency* și este dependent de sarcină (*task-dependent*).
- Exemplu: După ce bem cafeaua și devenim cafeinizați, vrem să citim o carte.
- În acest caz, selectăm cartea sub control **cognitiv și volitional**.
- Această formă de atenție este mai deliberată, bazată pe **criterii de selecție variabile** și este mai puternică, implicând efortul voluntar al subiectului (Figura 2).

Transpunerea în Mecanismele de Atenție Computaționale (Q-K-V)

- Mecanismele de atenție în *deep learning* sunt inspirate de aceste elemente volitional și nonvolitional.
- În cazul simplu, unde sunt disponibile doar elemente nonvolitionale (saliency), putem folosi un strat complet conectat parametrizat sau chiar o simplă operație de *max* sau *average pooling* pentru a aplica o selecție părtinitoare (*bias selection*) asupra *input*-urilor senzoriale.
- Ceea ce diferențiază mecanismele de atenție de straturile complet conectate sau de straturile de *pooling* este **includerea elementelor volitionale (volitional cues)**.
- **Mapping-ul Q-K-V:**

    - **Elementele Volitionale** sunt denumite **Queries ($\mathbf{q}$)**.
    - **Input-urile Senzoriale** (datele) sunt denumite **Values ($\mathbf{v}$)**.
    - Fiecare *value* este asociat cu un **Key ($\mathbf{k}$)**, care poate fi considerat elementul **nonvolitional cue** al acelui *input* senzorial.
- **Attention Pooling:** Mecanismul de *attention pooling* este proiectat astfel încât *query*-ul (elementul volitional) să interacționeze cu *key*-urile (elementele nonvolitionale), ghidând selecția părtinitoare asupra *values*-urilor (input-urilor senzoriale).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Saliency (Proeminența)

- **Analogie Simplificată (Panou Publicitar):** Saliency este ca un panou publicitar care se aprinde brusc. Chiar dacă ești concentrat pe volan (sarcină volitională), culoarea puternică, lumina intermitentă sau contrastul (proprietăți intrinseci) îți atrag **automat** atenția (nonvolitional).

Concept Dificil: Fovea și Macula

- **Context:** Sursa menționează că atenția aduce **fovea** (centrul maculei, unde acuitatea vizuală este cea mai mare) pe obiectul de interes.
- *Asta se întâmplă deoarece:* Fovea conține cea mai mare densitate de receptori vizuali (conuri) și este regiunea responsabilă de vederea detaliată. Prin mutarea foveei pe țintă, creierul alocă efectiv **resursa de cea mai înaltă rezoluție** către elementul care a fost selectat prin elementul volitional sau nonvolitional.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația dintre Cues și QKV:**
    - **Nonvolitional Cues (Saliency)** se mapează direct pe **Keys ($\mathbf{k}$)**: Acestea sunt proprietăți intrinseci ale datelor (ex: contrastul, forma).
    - **Volitional Cues (Task-Dependent)** se mapează direct pe **Queries ($\mathbf{q}$)**: Acestea reprezintă **ce caută** rețeaua la un moment dat.
    - **Input-urile Senzoriale** se mapează pe **Values ($\mathbf{v}$)**: Acestea sunt datele brute sau reprezentările *feature* intermediate care trebuie ponderate.
- **Implicația Cheie (Pooling vs. Attention):** *Pooling* simplu (Max sau Average) poate realiza doar o selecție bazată pe *saliency* (nonvolitional). Mecanismele de atenție se diferențiază prin includerea *queries*-urilor (volitional cues), permițând modelului să ia decizii de filtrare bazate pe **sarcină**.

5. EXERCITII (Conceptual și de Analiză)

Deoarece această secțiune este preponderent conceptuală, exercițiile se vor axa pe clasificarea și înțelegerea modelului QKV.

Exercițiul A.1: Clasificarea Elementelor de Atenție

Clasificați următoarele situații ca fiind dominant **Volitional (V)** sau **Nonvolitional (NV)**, conform teoriei atenției:

1. O persoană aude o alarmă puternică (sunetul este neașteptat și foarte intens).
2. Un student citește o carte de curs, căutând cuvântul cheie "Transformer" în text.
3. O rețea neuronală Seq2Seq folosește starea ascunsă a decodificatorului la momentul t pentru a determina ce cuvinte din propoziția sursă sunt relevante.

Rezolvare A.1:

1. **NV (Nonvolitional):** Intensitatea și caracterul neașteptat atrag atenția automat (saliency).
2. **V (Volitional):** Căutarea unui cuvânt cheie este o acțiune conștientă și dependentă de sarcină (*task-dependent*).
3. **V (Volitional):** Starea ascunsă a decodificatorului acționează ca un **Query** (element volitional), indicând ce informație este necesară pentru a îndeplini sarcina de decodificare la pasul curent.

Exercițiul A.2: Relația dintre Q-K-V și Biologie

Asociați termenii QKV din Mecanismul de Atenție cu echivalenții lor biologici/funcționali prezentați în sursă:

Termen Mecanism de Atenție	Echivalent Biologic/Funcțional
Query ($\mathbf{q}$)	
Key ($\mathbf{k}$)	
Value ($\mathbf{v}$)	

Rezolvare A.2:

Termen Mecanism de Atenție	Echivalent Biologic/Funcțional
Query ($\mathbf{q}$)	**Volitional cue** (Control cognitiv, dependent de sarcină)
Key ($\mathbf{k}$)	**Nonvolitional cue** (Saliency/Proeminența)
Value ($\mathbf{v}$)	**Sensory inputs** (Input-uri senzoriale/date)

Exercițiul A.3: Analiza Output-ului (Conceptual)

De ce este insuficientă utilizarea unui strat *fully-connected* sau a unui strat de *pooling* simplu pentru a implementa conceptul complet al atenției, conform sursei?

Rezolvare A.3:

Straturile *fully-connected* sau straturile de *pooling* (Max/Average) pot aplica doar o **selecție părtinitoare bazată pe *saliency*** (elemente nonvolitionale). Ceea ce face ca mecanismele de atenție să fie distincte și puternice este **includerea elementelor volitionale (queries)**. Fără un *query*, modelul nu poate lua decizii de filtrare care sunt dependente de sarcina specifică (adică, **ce caută** rețeaua în acel moment).

5.2

Această lecție, **5.2 Attention pooling: Nadaraya-Watson kernel regression**, oferă o perspectivă fundamentală asupra mecanismului de Atenție, ancorându-l într-un model statistic clasic pentru a oferi o înțelegere intuitivă și matematică a modului în care funcționează *Attention Pooling* în practică.

Ghid de Studiu Complet: 5.2 Attention Pooling: Regresia Kernel Nadaraya-Watson

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5.2 detaliază mecanismul de **Attention Pooling** folosind ca exemplu modelul de **Regresie Kernel Nadaraya-Watson**, propus în **1964**. Deși Nadaraya-Watson este un model statistic **nonparametric**, acesta ilustrează perfect principiul de bază al atenției: agregarea selectivă a *valorilor* (input-uri senzoriale) pentru a produce ieșirea. Cheia este modul în care interacțiunile dintre *queries* (elemente voliționale) și *keys* (elemente nonvoliționale) rezultă în **ponderi de atenție**. În esență, *Attention Pooling* nu este altceva decât o **medie ponderată a valorilor**, unde ponderile sunt determinate de relevanța *key*-ului față de *query*.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Principiul Atenției:** *Attention Pooling* este o medie ponderată selectivă a valorilor (y_i), unde ponderea $\alpha(x, x_i)$ este o distribuție de probabilitate validă.
2. **Modelul Ilustrativ:** Regresia Kernel Nadaraya-Watson (1964) este un exemplu simplu și complet de *machine learning* cu mecanisme de atenție.
3. **Maparea Q-K-V:** În acest model, x (noul input) este **Query**, perechea (x_i, y_i) este perechea **Key-Value**, unde x_i este **Key**-ul, iar y_i este **Value**-ul.
4. **Dependența de Distanță:** Dacă se folosește un **Kernel Gaussian**, ponderile de atenție $\alpha(x, x_i)$ sunt calculate folosind o funcție *softmax* a distanței negative pătratice dintre *query* (x) și *key* (x_i), asigurând că *key*-urile mai apropiate primesc mai multă atenție.

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.2 Attention pooling: Nadaraya-Watson kernel regression

Cadrul General al Regresiei

- Se consideră o problemă de regresie: dat un set de date *input-output* $(x_1, y_1), \ldots, (x_n, y_n)$, se dorește învățarea unei funcții f pentru a prezice ieșirea $\hat{y} = f(x)$ pentru orice nou *input* x.

Estimatorul de Bază (Average Pooling)

- Cel mai simplu estimator ("cel mai neinteligent estimator din lume") este **Average Pooling**:

$$f(x) = \frac{1}{n} \sum_{i=1}^{n} y_i \quad (1)$$

- Acest estimator omite complet *input*-urile x_i.

Regresia Kernel Nadaraya-Watson (1964)

- Nadaraya și Watson au propus o idee mai bună: ponderarea ieșirilor y_i în funcție de locațiile lor de intrare (x_i).
- Formula utilizează un **kernel** K (care măsoară similaritatea sau proximitatea):

$$f(x) = \sum_{i=1}^{n} \frac{K(x - x_i)}{{}_{=1}^{n} K(x - x)} y_i \quad (2)$$

Forma Generalizată de Attention Pooling

- Ecuația (2) poate fi rescrisă într-o formă mai generalizată de *attention pooling* (similar cu Figura 3 din cadrul teoretic):

$$f(x) = \sum_{i=1}^{n} \alpha(x, x_i) y_i \quad (3)$$

  - x este **query**-ul.
  - (x_i, y_i) este perechea **key-value**.
  - $\alpha(x, x_i)$ este **attention weight** (ponderea de atenție).
  - Ponderile de atenție α formează o **distribuție de probabilitate validă** (sunt non-negative și suma lor este 1).

Utilizarea Kernelului Gaussian (Demonstrație Softmax)

- Pentru a obține o intuiție despre *attention pooling*, se consideră un **kernel Gaussian** definit ca $K() = \frac{1}{\sqrt{2}}\exp(-\frac{^2}{2})$.
- Înlocuirea kernelului Gaussian în formula (2) rezultă în:

$$f(x) = \sum_{i=1}^{n} \frac{\exp(-\frac{1}{2}(x - x_i)^2)}{\sum_{=1}^{n} \exp(-\frac{1}{2}(x - x)^2)} y_i$$

- Această formulă este echivalentă cu aplicarea funcției **Softmax** pe funcția de aliniere negativă bazată pe distanța pătratică dintre x și x_i:

$$f(x) = \sum_{i=1}^{n} \text{softmax} \ -\frac{1}{2}(x - x_i)^2 \ y_i \quad (4)$$

- **Implicație:** Un *key* x_i care este **mai aproape** de *query* x va primi **mai multă atenție** via o pondere $\alpha(x, x_i)$ mai mare.

Atenția Nonparametrică vs. Parametrică

- Modelul Nadaraya-Watson kernel regression este un model **nonparametric**; prin urmare, Ecuația (4) este un exemplu de **nonparametric attention pooling**.
- Modelele nonparametric Nadaraya-Watson beneficiază de **consistență**: având suficiente date, modelul converge la soluția optimă.
- Se pot integra **parametri învățabili** în *attention pooling*. De exemplu, distanța dintre *query* și *key* poate fi înmulțită cu un parametru învățabil :

$$f(x) = \sum_{i=1}^{n} \text{softmax} \ -\frac{1}{2}((x - x_i))^2 \ y_i \quad (9)\text{simpifiată}$$

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Attention Weight (α) ca Distribuție de Probabilitate

- **Analogie (Împărțirea Bugetului):** Faptul că ponderile de atenție trebuie să fie non-negative și să însumeze 1 înseamnă că modelul are un "buget de atenție" de 100%. Dacă alocă 70% din atenție unui singur *key-value* specific, restul de 30% trebuie distribuit între toate celelalte *key-value*-uri rămase. Bugetul total de atenție este întotdeauna cheltuit.

Concept Dificil: Nonparametric vs. Parametric Attention Pooling

- **Explicație Simplificată (Ajustarea Manuală vs. Învățarea Automată):**

- **Nonparametric (Nadaraya-Watson):** Modelul folosește datele brute și o formulă fixă (kernel Gaussian) pentru a determina similaritatea. Similaritatea este rigidă și nu se modifică în timpul antrenamentului. Este ca și cum ai măsura distanța cu o riglă standard.
- **Parametric (cu învățabil):** Prin introducerea unui parametru , rețeaua poate învăța să **ajusteze** cât de importantă este acea distanță. Dacă este mare, distanța este amplificată (atenția devine foarte selectivă). Dacă este mic, distanța este estompată (atenția devine mai largă). Acest parametru permite modelului să învețe *cum* să își măsoare similaritatea, nu doar să o calculeze.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Pooling vs. Atenție):**
  - **CAUZĂ:** Estimatorul de bază (Average Pooling) ignoră complet x_i (locațiile de intrare).
  - **EFECT:** Regresia Nadaraya-Watson (Attention Pooling) rezolvă această problemă prin utilizarea funcției kernel K sau a Softmax-ului bazat pe distanța dintre *query* (x) și *key* (x_i), asigurând că *values* (y_i) sunt ponderate în funcție de proximitatea lor față de *query*.
- **Conexiunea cu Secțiunea 5 (QKV):** Acest model statistic validează cadrul QKV. Deși modelul este simplu, el demonstrează că mecanismul de atenție se reduce la calculul ponderilor de aliniere (date de distanța în kernel) și aplicarea lor pe *values*.

---

5. EXERCITII (Calcul bazat pe Regresia Kernel Nadaraya-Watson)

Voi genera un exercițiu numeric concentrat pe aplicarea formulei de atenție nonparametrică bazată pe Kernelul Gaussian (Ecuația 4).

Exercițiul N.1: Calculul Attention Pooling (Nonparametric)

Avem un set de date unidimensionale (x_i, y_i) cu $n = 3$ puncte. Se dorește prezicerea $f(x)$ pentru *query*-ul $x = 5$.

- **Query:** $x = 5$.
- **Date (Key-Value):**
  - $(_1, _1) = (4, 10)$
  - $(_2, _2) = (6, 5)$
  - $(_3, _3) = (10, 2)$

Se folosește formula de atenție nonparametrică (Ecuația 4), bazată pe Kernelul Gaussian, care calculează ponderea α_i ca:

$$\alpha_i = \text{softmax} \ -\frac{1}{2}(x - x_i)^2$$

Aproximări necesare:

- $\exp(-0.5) \approx 0.607$
- $\exp(-1.0) \approx 0.368$
- $\exp(-12.5) \approx 0.0000037$

1. Calculați scorurile de aliniere negative $a_i = -\frac{1}{2}(x - x_i)^2$.
2. Calculați ponderile de atenție $\alpha_1, \alpha_2, \alpha_3$ (utilizați aproximările).
3. Calculați output-ul final $f(x) = {}_{i=1}^{3} \alpha_i y_i$.

Rezolvare N.1:

1. **Calculul Scorurilor de Aliniere (a_i):**
   - $a_1 = -\frac{1}{2}(5-4)^2 = -\frac{1}{2}(1)^2 = \mathbf{-0.5}$
   - $a_2 = -\frac{1}{2}(5-6)^2 = -\frac{1}{2}(-1)^2 = \mathbf{-0.5}$
   - $a_3 = -\frac{1}{2}(5-10)^2 = -\frac{1}{2}(-5)^2 = -\frac{25}{2} = \mathbf{-12.5}$
2. **Calculul Ponderilor de Atenție (α_i):**
   - Termeni exponențiali:
     - $\exp(a_1) \approx 0.607$
     - $\exp(a_2) \approx 0.607$
     - $\exp(a_3) \approx 0.0000037$
   - Denominator Softmax (D): $0.607 + 0.607 + 0.0000037 \approx 1.214$
   - $$\alpha_1 = \frac{0.607}{1.214} \approx \mathbf{0.500}$$
   - $$\alpha_2 = \frac{0.607}{1.214} \approx \mathbf{0.500}$$
   - $$\alpha_3 = \frac{0.0000037}{1.214} \approx \mathbf{0.000}$$

     (Atenția este neglijabilă)
3. **Calculul Output-ului Final $f(x)$:**

$$f(5) = \alpha_1 y_1 + \alpha_2 y_2 + \alpha_3 y_3$$

$$f(5) \approx (0.500 \cdot 10) + (0.500 \cdot 5) + (0.000 \cdot 2)$$

$$f(5) \approx 5.0 + 2.5 + 0 = \mathbf{7.5}$$

Implicație: Deoarece *query*-ul $x = 5$ este egal depărtat de $x_1 = 4$ și $x_2 = 6$, atenția este distribuită egal între valorile lor ($y_1 = 10$ și $y_2 = 5$).

5.3

Suntem acum în fața lecției **5.3 Funcțiile de Scorerare a Atenției (Attention scoring functions)**. Această secțiune este fundamentală, deoarece explică modul în care *query*-ul și *key*-urile interacționează pentru a produce un **scor de aliniere** și introduce două metode practice și extrem de populare pentru calcularea acestui scor: Atenția Aditivă și Atenția Produsului Scalar Scalat.

Ghid de Studiu Complet: 5.3 Funcțiile de Scorerare a Atenției

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5.3 se concentrează pe mecanismul matematic de bază al atenției: **funcția de scorare** $a(\mathbf{q}, \mathbf{k})$. Această funcție, denumită și *attention scoring function*, transformă interacțiunea dintre *query* ($\mathbf{q}$) și *key* ($\mathbf{k}$) într-un scalar de relevanță. Acest scor este apoi alimentat într-o operație **Softmax** pentru a produce **ponderile de atenție** ($\alpha(\mathbf{q}, \mathbf{k})$). Output-ul final al *attention pooling*-ului este pur și simplu o **sumă ponderată a valorilor**, care acționează ca o medie ponderată. Secțiunea prezintă două alegeri majore pentru funcția de scorare a: **Atenția Aditivă** (potrivită când Q și K au lungimi diferite, folosind o transformare neliniară) și **Atenția Produsului Scalar Scalat** (mai eficientă computațional, care necesită lungimi egale, dar necesită scalare pentru stabilitate).

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Rolul Funcției de Scorarare (a):** Funcția $a(\mathbf{q}, \mathbf{k})$ transformă interacțiunea dintre *query* și *key* într-un scor scalar.
2. **Mecanismul de lețire:** Scorurile de aliniere sunt transformate prin Softmax în ponderi de atenție (α), iar output-ul final este o **sumă ponderată** (sau medie ponderată) a *values*-urilor.
3. **Atenția Aditivă (Additive Attention):** Este folosită atunci când vectorii *query* și *key* au **lungimi diferite** ($q \neq k$), folosind o rețea MLP cu un singur strat ascuns și $\tanh$ ca activare.
4. **Atenția Produsului Scalar Scalat (Scaled Dot-Product Attention):** Este mai eficientă computațional și este folosită când *query* și *key* au aceeași lungime ($q = k = d$), necesitând împărțirea la $\sqrt{d}$ pentru a menține varianța produsului scalar la 1, asigurând stabilitatea Softmax.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.3 Funcțiile de Scorarare a Atenției

Cadrul General al Atenției

- Secțiunea anterioară a utilizat exponentul kernelului Gaussian ca funcție de scorare a atenției.
- Rezultatele acestei funcții (scorurile de aliniere) sunt introduse în operația Softmax.
- Softmax-ul produce o **distribuție de probabilitate** (ponderile de atenție α) peste *values*-urile asociate cu *keys*-urile.
- Output-ul final al *attention pooling*-ului este o **sumă ponderată a *values*-urilor** bazată pe aceste ponderi de atenție. Această sumă ponderată este, în esență, o **medie ponderată**.

Formularea Matematică (Ecuațiile 5 și 6)

- Se presupune că avem un *query* $\mathbf{q} \in \mathbb{R}^q$ și m perechi *key-value* $(\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)$, unde $\mathbf{k}_i \in \mathbb{R}^k$ și $\mathbf{v}_i \in \mathbb{R}$.
- **Attention Pooling Output (f) (Ecuația 5):**

$$f(\mathbf{q}, (\mathbf{k}_1, \mathbf{v}_1), \ldots, (\mathbf{k}_m, \mathbf{v}_m)) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i \in \mathbb{R}$$

- **Ponderea de Atenție (α) (Ecuația 6):**

$$\alpha(\mathbf{q}, \mathbf{k}_i) = \mathrm{softmax}(a(\mathbf{q}, \mathbf{k}_i)) = \frac{\exp(a(\mathbf{q}, \mathbf{k}_i))}{\sum_{=1}^{m} \exp(a(\mathbf{q}, \mathbf{k}))} \in \mathbb{R}$$

Tipuri de Funcții de Scorarare (a)

1. Atenția Aditivă (Additive Attention)

- Utilizată în general când vectorii *query* și *key* au **lungimi diferite** ($q \neq k$).
- Funcția de scorare este definită ca (Ecuația 7):

$$(\mathbf{q}, \mathbf{k}) = \mathbf{w}_\mathbf{v}^\top \tanh(\mathbf{W}_\mathbf{q}\mathbf{q} + \mathbf{W}_\mathbf{k}\mathbf{k}) \in \mathbb{R}$$

- **Parametrii învățabili:** $\mathbf{W}_q \in \mathbb{R}^{h \times q}$, $\mathbf{W}_k \in \mathbb{R}^{h \times k}$, și $\mathbf{w}_v \in \mathbb{R}^h$.
- Această formulă este echivalentă cu un MLP cu un singur strat ascuns de dimensiunea h, care folosește $\tanh$ ca funcție de activare.

2. Atenția Produsului Scalar Scalat (Scaled Dot-Product Attention)

- Este un design mai eficient computațional, care folosește simplu **produsul scalar**.
- Operația produsului scalar necesită ca vectorii *query* și *key* să aibă **aceeași lungime**, notată d ($q = k = d$).

- Dacă elementele lui $\mathbf{q}$ și $\mathbf{k}$ sunt variabile aleatoare independente cu medie zero și varianță unitară, produsul scalar $\mathbf{q}^\top\mathbf{k}$ are medie zero și o **varianță de** d.
- **Necesitatea Scalării:** Pentru a asigura că varianța produsului scalar rămâne aproape de 1, indiferent de lungimea vectorului (d), produsul scalar este **împărțit la** $\sqrt{d}$.
- **Formula de Scorare (Ecuația 13):**

$$(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top\mathbf{k}/\sqrt{}$$

- **Calculul în Mini-Batch-uri:** Pentru eficiență, atenția este calculată pe mini-batch-uri:

$$\text{softmax}\ \frac{\top}{\sqrt{}}\ \in \mathbb{R}^{n\times}$$

unde $Q \in \mathbb{R}^{n\times d}$, $K \in \mathbb{R}^{m\times d}$, și $\in \mathbb{R}^{m\times v}$.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Functia de Scorarare (a)

- **Analogie Simplificată (Test de Compatibilitate):** Funcția $a(\mathbf{q}, \mathbf{k})$ este ca un **test de compatibilitate** între un cumpărător ($\mathbf{q}$) și un produs ($\mathbf{k}$). Cu cât scorul a este mai mare, cu atât sunt mai compatibile sau aliniate (relevant *key* pentru *query*). Acest scor scalar brut determină cât de mult din bugetul de atenție va primi acel *key* după normalizarea Softmax.

Concept Dificil: Produs Scalar vs. Atenție Aditivă

- **Atenția Produsului Scalar:** Este mai rapidă, deoarece folosește o operație liniară simplă (înmulțirea matricială/produsul scalar). Este preferată în arhitecturile moderne (ca Transformer) pentru viteză.
- **Atenția Aditivă:** Este mai expresivă, folosind neliniarități ($\tanh$) și o rețea MLP. Acest lucru o face potrivită pentru cazurile în care *query* și *key* au dimensiuni diferite (ceea ce Produsul Scalar nu poate gestiona direct).

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Scalare și Stabilitate):**
  - **CAUZĂ:** Pe măsură ce dimensiunea d a vectorilor *query* și *key* crește, varianța produsului scalar $\mathbf{q}^\top\mathbf{k}$ crește proporțional cu d.
  - **EFECT:** Introducerea unor valori mari în funcția Softmax duce la **gradienți extrem de mici** (Softmax-ul devine "încremenit" cu o distribuție polarizată). **Scalarea cu**

$\sqrt{d}$ menține varianța constantă, prevenind instabilitatea numerică în timpul antrenamentului.

- **Conexiunea cu Secțiunea 5.2:** Functia de scorare bazată pe kernelul Gaussian utilizată anterior este, de asemenea, o formă de $a(\mathbf{q}, \mathbf{k})$, care a servit drept punte către formele parametrice introduse aici.

---

5. EXERCITII (Calcul pe Funcțiile de Scorarare a Atenției)

Voi genera un exercițiu de calcul bazat pe structura din **05_08_Exercises.pdf** (Exercițiul 5), concentrându-mă pe ambele tipuri de atenție.

Set de Date (Modificat pentru Examen):

- $m = 2$ perechi *key-value*. $k = 2, v = 2$.
- Query: $\mathbf{q} = [1\ 0]$.
- Perechile Key-Value:

$$(\mathbf{k}_1, \mathbf{v}_1) = ([1\ 1], [2\ 0]), \quad (\mathbf{k}_2, \mathbf{v}_2) = ([-1\ 1], [0\ 4])$$

Exercițiul A.4: Atenția Produsului Scalar Scalat (Calcul)

Utilizați setul de date de mai sus. Calculați output-ul final al *Attention Pooling* folosind formula de **Atenție Produs Scalar Scalat**.

- $d = k = 2$. (Factorul de scalare este $1/\sqrt{2} \approx 0.707$).
- Presupuneri pentru Softmax: $\exp(0.707) \approx 2.028, \exp(0) = 1.0$.

1. Calculați scorurile de aliniere $a(\mathbf{q}, \mathbf{k}_1)$ și $a(\mathbf{q}, \mathbf{k}_2)$.
2. Calculați ponderile de atenție α_1 și α_2 (utilizați aproximările date).
3. Calculați output-ul final $f(\mathbf{q}, \ldots)$.

Rezolvare A.4:

1. **Calculul Scorurilor de Aliniere ($a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}/\sqrt{d}$):**
   - $d = 2 \quad 1/\sqrt{d} \approx 0.707$.
   - $$a(\mathbf{q}, \mathbf{k}_1) = ([1 \quad 0]\,[1\ 1]) \cdot 0.707 = (1) \cdot 0.707 = \mathbf{0.707}$$
   - $$a(\mathbf{q}, \mathbf{k}_2) = ([1 \quad 0]\,[-1\ 1]) \cdot 0.707 = (-1) \cdot 0.707 = \mathbf{-0.707}$$
2. **Calculul Ponderilor de Atenție (α):**
   - Termeni exponențiali: $\exp(0.707) \approx 2.028$ și $\exp(-0.707) \approx 0.493$.
   - Denominator Softmax (D): $2.028 + 0.493 \approx 2.521$.

- $$\alpha_1 = \frac{2.028}{2.521} \approx \mathbf{0.804}$$
- $$\alpha_2 = \frac{0.493}{2.521} \approx \mathbf{0.196}$$

3. **Calculul Output-ului Final ($f(\mathbf{q}, \ldots)$):**

$$f(\mathbf{q}, \ldots) = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2$$
$$f(\mathbf{q}, \ldots) \approx 0.804\,[2\ 0] + 0.196\,[0\ 4]$$
$$f(\mathbf{q}, \ldots) \approx [1.608\ 0] + [0\ 0.784] = [1.608\ 0.784]$$

Exercițiul A.5: Analiza Dimensionalității și Parametrilor (Atenția Aditivă)

Presupuneți că se folosește **Atenția Aditivă**. *Query*-urile au dimensiunea $q = 5$, iar *Keys*-urile au dimensiunea $k = 8$. Dimensiunea stratului ascuns intern este $h = 10$.

1. Calculați dimensiunea matricilor de ponderi $\mathbf{W}_q$ și $\mathbf{W}_k$.
2. Determinați numărul total de parametri învățabili necesari pentru funcția de scorare $a(\mathbf{q}, \mathbf{k})$.

Rezolvare A.5:

1. **Dimensiunea Matricilor de Ponderi:**
   - $\mathbf{W}_q \in \mathbb{R}^{h \times q} \quad \mathbf{W}_q \in \mathbb{R}^{10 \times 5}$
   - $\mathbf{W}_k \in \mathbb{R}^{h \times k} \quad \mathbf{W}_k \in \mathbb{R}^{10 \times 8}$
   - $\mathbf{w}_v \in \mathbb{R}^{h} \quad \mathbf{w}_v \in \mathbb{R}^{10}$
2. **Numărul Total de Parametri Învățabili:**

$$\text{Parametri} = (10 \times 5) + (10 \times 8) + 10$$
$$\text{Parametri} = 50 + 80 + 10 = \mathbf{140}$$

5.4

Absolut! Lecția **5.4 Bahdanau Attention** este esențială deoarece marchează momentul în care mecanismele de atenție au fost integrate în arhitectura Codificator-Decodificator, rezolvând o limitare majoră a primelor modele *Sequence-to-Sequence* bazate pe RNN.

Ghid de Studiu Complet: 5.4 Bahdanau Attention

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5.4 detaliază modelul de atenție propus de **Bahdanau et al.** (2015), o soluție revoluționară pentru a depăși **limitarea severă de aliniere** a arhitecturilor clasice Codificator-Decodificator RNN (discutate în Secțiunea 4.6). În modelul clasic Seq2Seq, întreaga secvență de intrare (sursă) era condensată într-o singură variabilă de context (c) de formă fixă, care era utilizată la **fiecare pas de decodificare**. Acest lucru crea un "gât de sticlă" informațional, mai ales pentru secvențele lungi. Atenția Bahdanau rezolvă această problemă permițând modelului să **alineze selectiv** și să acorde atenție doar părților relevante din secvența de intrare (sursă) la fiecare pas de predicție (decodificare). Acest lucru este realizat prin tratarea variabilei de context ($c_{t'}$) ca fiind **output-ul *attention pooling*-ului**, calculat dinamic la fiecare pas de timp t' al decodificatorului.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Problema Rezolvată:** Atenția Bahdanau elimină limitarea conform căreia aceeași variabilă de context de formă fixă (c) trebuie folosită la fiecare pas de decodificare.
2. **Mecanismul Q-K-V specific:** *Query*-ul ($\mathbf{q}$) este starea ascunsă anterioară a decodificatorului ($\mathbf{s}_{t'-1}$), iar *Keys* și *Values* sunt stările ascunse ale codificatorului la toți pașii de timp ($\mathbf{h}_t$).
3. **Funcția de Scorarare:** Bahdanau folosește **Atenția Aditivă** (*additive attention*) pentru a calcula scorurile de aliniere $\alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t)$.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.4 Bahdanau Attention

Contextul Problemei Sequence-to-Sequence

- În Secțiunea 4.6, s-a studiat problema traducerii automate folosind o arhitectură Codificator-Decodificator bazată pe două RNN-uri.
- Codificatorul RNN transformă o secvență de lungime variabilă într-o variabilă de context de formă fixă.
- Decodificatorul RNN generează secvența țintă (*token* cu *token*) pe baza *token*-urilor generate anterior și a variabilei de context.
- **Limita Majoră:** Deși nu toți *token*-ii de intrare sunt utili pentru decodificarea unui anumit *token* de ieșire, **aceeași** variabilă de context care codifică întreaga secvență de intrare este utilizată la fiecare pas de decodificare.

Soluția Bahdanau (Context Variabil Dinamic)

- Modelul Bahdanau propune un model de atenție diferențiabilă, inspirat de ideea de a învăța alinierea fără limitarea severă a alinierei unidirecționale.
- Obiectivul este ca modelul să se alinieze (sau să acorde atenție) doar părților din secvența de intrare care sunt **relevante** pentru predicția curentă.

- Acest lucru este realizat prin tratarea variabilei de context c ca fiind **output-ul *attention pooling*-ului**.
- În arhitectura bazată pe atenție, variabila de context c este înlocuită cu o variabilă dinamică $c_{t'}$ la fiecare pas de decodificare t'.

Formula Atenției Bahdanau (Attention Pooling Output)

- Starea ascunsă a decodificatorului la pasul t' utilizează contextul dinamic $c_{t'}$:

$$_{t'} = \sum_{t=1} \alpha(\mathbf{s}_{t'-1}, \mathbf{h}_t)\mathbf{h}_t$$

  - este numărul de *token*-uri din secvența de intrare.

Maparea Q-K-V specifică Bahdanau:

- **Query ($\mathbf{q}$):** Este **starea ascunsă anterioară a decodificatorului** ($\mathbf{s}_{t'-1}$) la pasul de timp $t'-1$. (Aceasta reprezintă elementul volitional/dependent de sarcină).
- **Keys și Values ($\mathbf{k}_t, \mathbf{v}_t$):** Sunt **stările ascunse ale codificatorului** la toți pașii de timp t ($\mathbf{h}_t$).
- **Scorul de Aliniere (α):** Ponderile de atenție α sunt calculate folosind **funcția de scorare a atenției aditive** (*additive attention scoring function*), definită anterior (Ecuația 7 din Secțiunea 5.3).

Arhitectura (Figura 5)

- Arhitectura Codificator-Decodificator RNN cu Atenție Bahdanau este similară cu arhitectura RNN *vanilla* (Secțiunea 4.6), cu o mică diferență: starea ascunsă a decodificatorului este folosită pentru a calcula atenția, care apoi alimentează stratul recurent și stratul FC de ieșire.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Context Variabil Dinamic ($c_{t'}$) în Seq2Seq

- **Analogie Simplificată (Traducătorul Uman):** Modelul RNN *vanilla* era ca un traducător uman care citește toată propoziția sursă o singură dată și apoi încearcă să o traducă, bazându-se doar pe acea primă impresie (vectorul c fix).
- **Atenția Bahdanau** este ca un traducător care, la momentul traducerii cuvântului t' (Query), se uită **dinamic** înapoi la cuvintele sursă cele mai relevante (Keys/Values), asigurându-se că traducerea curentă se aliniază corect cu partea potrivită a propoziției originale.

Concept Dificil: Folosirea Stării Decodificatorului ca Query

- *Asta se întâmplă deoarece:* Starea ascunsă anterioară a decodificatorului ($\mathbf{s}_{t'-1}$) **rezumă** ceea ce a fost deja tradus până la acel moment. Prin urmare, $\mathbf{s}_{t'-1}$ este cel mai bun **element volitional (Query)** disponibil pentru a determina: "Ce informație din sursă (Keys) am nevoie acum pentru a genera următorul cuvânt în țintă?".

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Secțiunea 4.6 (Seq2Seq):** Atenția Bahdanau este o **îmbunătățire directă** a arhitecturii Seq2Seq. Ea înlocuiește vectorul de context static (c) cu un vector de context dinamic ($c_{t'}$), calculat la fiecare pas t'.
- **Conexiunea cu Secțiunea 5.3 (Funcții de Scorare):** Modelul Bahdanau a ales să folosească **Atenția Aditivă**. Acest lucru implică faptul că $\mathbf{s}_{t'-1}$ (Query) și $\mathbf{h}_t$ (Key) pot avea dimensiuni diferite, iar alinierea este calculată printr-o transformare neliniară (MLP cu $\tanh$).

---

5. EXERCITII (Calcul și Analiză bazate pe Bahdanau Attention)

Voi genera un exercițiu care combină calculul scorului Aditiv (specific Bahdanau) cu analiza componentelor Seq2Seq.

Exercițiul B.1: Calculul Scorului de Aliniere Aditivă (Inspired by Ex. 5 from Source)

Avem un sistem Bahdanau Attention. Se dorește calcularea scorului de aliniere pentru un singur *key* al codificatorului ($\mathbf{k}_1$) față de starea decodificatorului ($\mathbf{q}$). Se folosește formula atenției aditive.

- **Query (Starea Decodificatorului, $\mathbf{s}_{t'-1}$):** $\mathbf{q} = [1\ 0]$ (Dimensiune $q = 2$)
- **Key (Starea Codificatorului, $\mathbf{h}_1$):** $\mathbf{k}_1 = [2\ 1]$ (Dimensiune $k = 2$)
- **Dimensiunea stratului ascuns:** $h = 2$
- **Ponderi (generat aleatoriu):**

$$\mathbf{w}_v^\top = [1 \quad 1] \in \mathbb{R}^{1\times 2}, \quad \mathbf{W}_q = [1 \quad 0\ 0 \quad 1] \in \mathbb{R}^{2\times 2}, \quad \mathbf{W}_k = [1 \quad 0\ 0 \quad 1] \in \mathbb{R}^{2\times 2}$$

1. Calculați intrarea stratului $\tanh$: $\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}_1$.
2. Calculați scorul de aliniere $a(\mathbf{q}, \mathbf{k}_1)$ (rezultatul final trebuie să fie scalar).
   - Presupuneți $\tanh(3) \approx 0.995$.

Rezolvare B.1:

1. **Calculul Argumentului $\tanh$:**

$$\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}_1 = [1 \quad 0\,0 \quad 1]\,[1\;0] + [1 \quad 0\,0 \quad 1]\,[2\;1]$$
$$= [1\;0] + [2\;1] = [3\;1]$$

2. **Calculul Scorului de Aliniere $a(\mathbf{q}, \mathbf{k}_1)$:**

$$a(\mathbf{q}, \mathbf{k}_1) = \mathbf{w}_v^\top \tanh\left([3\;1]\right)$$
$$a(\mathbf{q}, \mathbf{k}_1) = [1 \quad 1]\,[\tanh(3)\;\;\tanh(1)]$$

   - Folosind aproximarea $\tanh(3) \approx 0.995$ și notând $\tanh(1) \approx 0.76$:

$$a(\mathbf{q}, \mathbf{k}_1) \approx (1 \cdot 0.995) + (1 \cdot 0.76) = 0.995 + 0.76 = \mathbf{1.755}$$

Exercițiul B.2: Impactul Atenției (Conceptual)

De ce se afirmă că, chiar dacă nu toți *token*-ii de intrare (sursă) sunt utili pentru decodificarea unui anumit *token* țintă, în arhitectura *vanilla* Seq2Seq, se folosește totuși **aceeași** variabilă de context? Explicați limitarea.

Rezolvare B.2:

Limita majoră a arhitecturii *vanilla* Seq2Seq (Secțiunea 4.6) este că variabila de context (c) este definită ca fiind **starea ascunsă finală a Codificatorului** ($\mathbf{h}$). Această variabilă:

1. **Este de formă fixă:** Indiferent cât de lungă este secvența de intrare, c are o dimensiune fixă.
2. **Este statică:** Odată calculată, **același** vector c este folosit ca intrare sau pentru inițializarea decodificatorului la **toți** pașii de decodificare.

Această limitare înseamnă că modelul nu poate filtra selectiv, fiind forțat să memoreze toată informația sursă într-un singur vector, ceea ce devine ineficient, în special pentru secvențele lungi, și duce la pierderea de informații relevante pe termen lung. Atenția Bahdanau rezolvă acest lucru prin calculul dinamic al $c_{t'}$.

5.5

Excelent! Lecția **5.5 Multi-head attention (Atenția Multi-Cap)** reprezintă o evoluție crucială a mecanismului de atenție, permițând modelului să capteze o gamă mai largă de dependențe.

Ghid de Studiu Complet: 5.5 Atenția Multi-Cap (Multi-head attention)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 5.5 introduce conceptul de **Atenție Multi-Cap (*Multi-head attention*)**, o tehnică de îmbunătățire a capacității de reprezentare a mecanismului de atenție standard. În practică, chiar și cu același set de *queries*, *keys* și *values*, este de dorit ca modelul să poată **combina cunoștințe provenite din comportamente diferite** ale aceluiași mecanism de atenție. De exemplu, ar putea fi necesar să fie capturate dependențe de diferite intervale (cum ar fi dependențe pe termen scurt vs. pe termen lung) dintr-o singură secvență. Soluția propusă de Atenția Multi-Cap este de a permite mecanismului de atenție să utilizeze în comun **subspații de reprezentare diferite** ale *queries*, *keys* și *values*.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul:** Permite modelului să combine cunoștințe din comportamente diferite, capturând dependențe de diverse intervale (scurt vs. lung) dintr-o secvență.
2. **Mecanismul de Proiecție:** În loc de o singură *attention pooling*, *queries*, *keys* și *values* sunt transformate cu h **proiecții liniare** învățate independent.
3. **Output-ul Final:** Ieșirile de la cele h *attention pooling outputs* (numite **capete** sau *heads*) sunt **concatenate** și apoi transformate printr-o altă proiecție liniară învățată pentru a produce ieșirea finală.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.5 Multi-head attention (Atenția Multi-Cap)

Motivație și Design

- În practică, se dorește ca modelul să combine cunoștințe din diferite comportamente ale mecanismului de atenție.
- Este benefic să se permită mecanismului de atenție să utilizeze în comun subspații de reprezentare diferite ale *queries*, *keys* și *values*.

Structura Multi-Cap (Multi-head)

- În loc să efectueze o singură *attention pooling*, *queries* ($\mathbf{q}$), *keys* ($\mathbf{k}$) și *values* ($\mathbf{v}$) sunt transformate cu h **proiecții liniare învățate independent**.
- Aceste h *queries*, *keys* și *values* proiectate sunt apoi introduse în *attention pooling* **în paralel**.
- Designul se numește **Atenție Multi-Cap**, unde fiecare dintre cele h ieșiri *attention pooling* este un **cap** (*head*).
- Straturi complet conectate (FC layers) sunt folosite pentru a realiza transformările liniare învățabile.

Calculul Ieșirii (Output) (Matematică și Flux)

- Fiecare cap de atenție i (unde $i = 1, \ldots, h$) este calculat ca:

$$\mathbf{h}_i = f(\mathbf{W}_i^{(q)}\mathbf{q}, \mathbf{W}_i^{(k)}\mathbf{k}, \mathbf{W}_i^{(v)}\mathbf{v}) \in \mathbb{R}^{P_v}$$

  - f este funcția de *attention pooling* (ex: atenția aditivă sau produsul scalar scalat, discutate în Secțiunea 5.3).
  - Parametrii $\mathbf{W}_i^{(q)}$, $\mathbf{W}_i^{(k)}$, și $\mathbf{W}_i^{(v)}$ sunt matrici învățabile, unice pentru fiecare cap i.
- **Finalizarea Output-ului:** La final, cele h ieșiri *attention pooling* ($\mathbf{h}_1, \ldots, \mathbf{h}_h$) sunt **concatenate**.
- Output-ul final al Atenției Multi-Cap este o altă **transformare liniară** a concatenării capetelor, folosind parametrii învățabili $\mathbf{W}_o \in \mathbb{R}^{P_o \times hp_v}$:

$$\text{tpt} = \mathbf{W}_o \begin{bmatrix} \mathbf{h}_1 & \mathbf{h}_h \end{bmatrix} \in \mathbb{R}^{P_o}$$

Beneficii

- Pe baza acestui design, fiecare cap poate acorda atenție **diferitelor părți ale *input*-ului**.
- Funcții mai sofisticate decât simpla medie ponderată pot fi exprimate.
- Figura 6 ilustrează vizual cum sunt concatenate multiplele capete și apoi transformate liniar.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Cap de Atenție (Head)

- **Analogie Simplificată (Echipa de Experți):** Atenția standard este ca un singur expert care examinează datele și oferă un singur răspuns. **Atenția Multi-Cap** este ca o echipă de h experți independenți.
  - Expertul 1 (Head 1) s-ar putea concentra pe dependențele de ordinul 2 (bigrame).
  - Expertul 2 (Head 2) s-ar putea concentra pe dependențele de lungă durată din propoziție.
  - Fiecare expert are propria sa "pereche de ochelari" (matricele $\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)}$) pentru a vedea datele într-un subspațiu de reprezentare diferit.

Concept Dificil: Proiecțiile Liniare Independente

- *Asta se întâmplă deoarece:* Dacă am folosi aceleași *queries*, *keys* și *values* inițiale pentru toate capetele, toate capetele ar calcula exact același rezultat. Prin **proiectarea liniară independentă** (înmulțirea cu $\mathbf{W}_i$ unice pentru fiecare cap), vectorul $\mathbf{q}$ original este transformat într-un vector $\mathbf{q}'$ care pune accentul pe *feature*-uri diferite (subspațiu diferit) pentru fiecare cap. Acest lucru garantează că fiecare cap învață un comportament de atenție distinct.

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Concatenarea):**
  - **CAUZĂ:** Fiecare cap generează o ieșire $\mathbf{h}_i$ care a capturat un anumit tip de dependență (ex: scurtă sau lungă).
  - **EFECT:** Ieșirile capetelor sunt **concatenate** (unite) pentru a forma un vector mai mare, care **combină cunoștințele** din toate cele h subspații de reprezentare. Transformarea liniară finală ($\mathbf{W}_o$) permite modelului să integreze aceste cunoștințe multiple într-o singură reprezentare coerentă.
- **Conexiunea cu Secțiunea 5.3:** Deși mecanismul Multi-Cap introduce proiecții, funcția de bază *attention pooling* f din interiorul fiecărui cap folosește metodele de scorare definite anterior, cum ar fi Atenția Aditivă sau Atenția Produsului Scalar Scalat.

---

5. EXERCITII (Calcul pe Arhitectura Multi-Cap)

Voi genera exerciții de calcul concentrate pe parametrizarea și fluxul de date într-o arhitectură Multi-Cap.

Exercițiul M.1: Calculul Parametrilor (Multi-head attention)

Calculați numărul total de parametri necesari pentru a implementa un strat **Multi-head attention** cu $h = 4$ capete.

- Dimensiunea de intrare Q, K, V: $d_q = d_k = d_v = 10$.
- Dimensiunea de proiecție pentru fiecare cap: $p_q = p_k = p_v = 5$.
- Dimensiunea de ieșire finală: $P_o = 10$.

Rezolvare M.1:

1. **Parametrii Proiecțiilor (Total pentru toate Capetele):** Fiecare cap i (din $h = 4$) are 3 matrici de ponderi independente: $\mathbf{W}_i^{(q)}, \mathbf{W}_i^{(k)}, \mathbf{W}_i^{(v)}$.
   - $^{(q)}$ (per cap): $p_q \times d_q = 5 \times 10 = 50$
   - $^{(k)}$ (per cap): $p_k \times d_k = 5 \times 10 = 50$
   - $^{(v)}$ (per cap): $p_v \times d_v = 5 \times 10 = 50$

$$\text{otaparametriperap(proiecții)} = 50 + 50 + 50 = 150$$

$$\text{otaparametriproiecții(4apete)} = 4 \times 150 = \mathbf{600}$$

2. **Parametrii Matricei de Ieșire ($\mathbf{W}_o$):**
   - Ieșirile capetelor sunt concatenate. Dimensiunea totală concatenată (intrarea în $\mathbf{W}_o$) este $h \times p_v$.

- Dimensiune intrare $\mathbf{W}_o$: $4 \times 5 = 20$.
- Dimensiune ieșire $\mathbf{W}_o$: $P_o = 10$.
- $\mathbf{W}_o \in \mathbb{R}^{P_o \times (h \cdot p_v)}$ $\quad 10 \times 20 = \mathbf{200}$.

3. **Total Parametri Multi-head attention:**

$$\text{ota} = 600(\text{proiecții}) + 200(\mathbf{W}_o) = \mathbf{800}$$

Exercițiul M.2: Fluxul de Date (Conceptual)

Explicați de ce concatenarea ieșirilor capetelor ($\mathbf{h}_1, \ldots, \mathbf{h}_h$) este urmată de o transformare liniară ($\mathbf{W}_o$) și nu este folosită direct ca ieșire finală.

Rezolvare M.2:

Transformarea liniară finală via $\mathbf{W}_o$ este esențială pentru a **integra cunoștințele multiple** capturate de fiecare cap.

1. **Integrare și Combinare:** Deși concatenarea adună rezultatele, $\mathbf{W}_o$ învață **cum să combine optim** informațiile din subspațiile disparate pe care le-a examinat fiecare cap.
2. **Controlul Dimensionalității:** Transformarea finală aduce dimensiunea înapoi la dimensiunea de ieșire dorită ($P_o = 10$ în exemplul anterior). Dacă nu ar exista $\mathbf{W}_o$, ieșirea ar avea o dimensiune mult mai mare (concatenată), ceea ce ar putea fi incompatibil cu straturile următoare ale rețelei.

5.6

Bună ziua! Lecția **5.6 Self-attention and positional encoding (Atenția Proprie și Codificarea Poziațională)** reprezintă o paradigmă radicală în modelarea secvențelor, mutând accentul de la procesarea secvențială (specifică RNN-urilor) la cea paralelă (specifică arhitecturilor moderne precum Transformer).

Ghid de Studiu Complet: 5.6 Atenția Proprie și Codificarea Poziațională

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Atenția proprie (*Self-attention*), numită și **Atenție Intra-secvențială (*intra-attention*)**, este un mecanism de codificare a secvențelor care folosește același set de *token*-uri din secvența de intrare pentru a acționa simultan ca **Queries, Keys și Values**. Această arhitectură revoluționară permite fiecărui *token* să fie conectat direct la orice alt *token* din secvență. Prin înlocuirea operațiilor secvențiale (cum ar fi cele din RNN-uri) cu operații paralele, atenția proprie atinge o **lungime maximă a căii de** $O(1)$, ceea ce facilitează mult învățarea dependențelor pe termen lung în cadrul secvenței. Totuși, deoarece procesarea este paralelă și nu secvențială, modelul pierde informația despre ordinea *token*-urilor. Soluția

este adăugarea **Codificării Poziționale (*positional encoding*)** la reprezentările de intrare pentru a injecta informații despre ordinea absolută sau relativă a secvenței.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Definiție Self-Attention:** Același set de *token*-uri acționează ca *queries*, *keys* și *values*.
2. **Avantaj Computațional:** Atenția proprie permite **paralelizarea completă** a calculului, având $O(1)$ operații secvențiale și cea mai scurtă lungime a căii, $O(1)$.
3. **Dezavantaj Computațional:** Complexitatea computațională este **pătratică** în raport cu lungimea secvenței, $O(n^2 d)$.
4. **Codificarea Poziațională:** Informațiile despre ordinea secvenței sunt injectate prin adăugarea unei matrici de codificare poziațională (P) la reprezentarea de intrare (X).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

5.6 Atenția Proprie și Codificarea Poziațională

Definiția Atenției Proprii (Self-attention / Intra-attention)

- În *deep learning*, CNN-urile sau RNN-urile sunt utilizate frecvent pentru a codifica o secvență.
- În atenția proprie, o secvență de *token*-uri este introdusă în *attention pooling*, astfel încât **același set de *token*-uri acționează ca *queries*, *keys* și *values***.
- Fiecare *query* acordă atenție tuturor perechilor *key-value* și generează o singură ieșire de atenție.
- Dacă secvența de intrare este $_1, \ldots, _n$ ($_i \in \mathbb{R}^d$), atenția proprie produce o secvență de aceeași lungime $_1, \ldots, _n$, unde $_i = f(_i, (_1, _1), \ldots, (_n, _n))$.

Comparație Arhitecturală (CNN vs. RNN vs. Self-attention)

Se compară arhitecturile în termeni de complexitate computațională, operații secvențiale și lungimea maximă a căii:

Arhitectură	Complexitate Computațională (pe Strat)	Operații Secvențiale	Lungime Maximă a Căii	Observații
CNN	$O(knd^2)$ (unde k = kernel size)	$O(1)$ (permite paralelism)	$O(n/k)$	Structură ierarhică.
RNN	$O(nd^2)$	$O(n)$ (nu poate fi paralelizat)	$O(n)$	Procesează recurent *token*-uri, unul câte unul.

Arhitectură	Complexitate Computațională (pe Strat)	Operații Secvențiale	Lungime Maximă a Căii	Observații
Self-attention	$O(n^2 d)$	$O(1)$ (permite paralelism)	$O(1)$ (cea mai scurtă)	Fiecare *token* conectat direct la oricare altul.

- **Concluzie:** Atât CNN-urile, cât și atenția proprie beneficiază de calcul paralel. Atenția proprie are cea mai scurtă lungime maximă a căii, dar complexitatea sa **pătratică** în raport cu lungimea secvenței ($O(n^2 d)$) o face prohibitiv de lentă pentru secvențe foarte lungi.

Codificarea Poziațională (Positional Encoding)

- Spre deosebire de RNN-uri care procesează *token*-urile secvențial, atenția proprie **elimină operațiile secvențiale** în favoarea calculului paralel.
- Acest lucru face ca modelul să piardă informațiile despre ordinea secvenței.
- Pentru a folosi informațiile despre ordinea secvenței, se injectează informații poziționale (absolute sau relative) prin **adăugarea Codificării Poziționale** la reprezentările de intrare.
- Codificările poziționale pot fi fie învățate, fie fixe.
- Se descrie o codificare poziațională fixă bazată pe funcții **sinus și cosinus**.
- Reprezentarea de intrare $X \in \mathbb{R}^{n \times d}$ (embedding-uri d-dimensionale pentru n *token*-uri) este adunată cu matricea de codificare poziațională $P \in \mathbb{R}^{n \times d}$.
- **Formula (Ecuația 9):** Elementul de pe rândul i și coloana 2 sau $2+1$ este dat de:

$$_{i,2} = \sin \frac{i}{10000^{2/d}} \quad \text{și} \quad _{i,2+1} = \text{os} \frac{i}{10000^{2/d}} \qquad (9)$$

  - i: indexul poziției.
  - 2 și $2+1$: indicii dimensionali (coloanele).
- Rândurile matricii P corespund pozițiilor, iar coloanele reprezintă diferite dimensiuni de codificare poziațională.
- Alternarea funcțiilor $\sin$ și os creează un decalaj. Coloanele 6 și 7 au o frecvență mai mare decât 8 și 9 (Figura 8).
- Această codificare permite modelului nu doar să capteze informații poziționale absolute, ci și să **învețe ușor să acorde atenție pozițiilor relative**.
- Matematic, codificarea la poziția $i+$ poate fi reprezentată ca o proiecție liniară a celei de la poziția i, unde matricea 2×2 de proiecție nu depinde de indexul poziției i.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Self-Attention (Intra-attention)

- **Explicație Simplificată (Autoreflectare):** Imaginați-vă că un cuvânt dintr-o propoziție, de exemplu "măr", vrea să știe la ce cuvinte ar trebui să se uite în acea propoziție pentru a fi înțeles corect. Cuvântul "măr" se trimite pe sine însuși ca *Query* (ce caut?). Trimite toate celelalte cuvinte ca *Keys* (ce oferiți?) și *Values* (care este conținutul vostru?). Dacă propoziția este "Eu mănânc un măr roșu", atenția proprie ajută "măr" să înțeleagă că "roșu" este cel mai relevant *key* pentru sensul său.

Concept Dificil: $O(1)$ Lungimea Căii Maxima

- *Asta se întâmplă deoarece:* În RNN-uri, pentru ca informația de la primul *token* (x_1) să ajungă la ultimul *token* (x_n), ea trebuie să treacă prin $n-1$ pași recurenți (lungimea căii $O(n)$). În Atenția Proprie, x_1 este direct conectat la x_n (și la toți ceilalți *token*-i) într-un singur strat de atenție (lungimea căii $O(1)$).
- **De ce este importantă calea scurtă?** O cale mai scurtă facilitează învățarea dependențelor pe distanțe lungi (long-range dependencies), deoarece gradientul nu trebuie să traverseze multe straturi secvențiale înainte de a ajunge la sursă, reducând problemele de dispariție/explozie a gradientului.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Paralelism vs. Ordine):**
  - **CAUZĂ:** Atenția proprie alege paralelizarea (timp de execuție rapid) și operațiile secvențiale $O(1)$.
  - **EFECT:** Modelul nu știe unde se află un *token* în secvență.
  - **SOLUȚIE:** Codificarea pozițională (P) devine obligatorie, injectând informația de ordine care altfel ar fi pierdută.
- **Conexiunea cu Secțiunea 5.3:** Atenția Produsului Scalar Scalat este adesea folosită în atenția proprie, deoarece este eficientă computațional și se bazează pe operații matriciale care se aliniază bine cu calculul pe mini-batch-uri ($O(n^2d)$).

---

5. EXERCITII (Analiză și Calcul)

Voi genera un exercițiu de comparare a complexității și un exercițiu de calcul bazat pe modelul de Atenție Aditivă din sursa 05_08_Exercises.pdf.

Exercițiul P.1: Comparație Computațională (RNN vs. Self-attention)

Comparați o arhitectură RNN cu un strat Self-attention în funcție de complexitatea computațională totală și lungimea maximă a căii, asumând $n = 1000$ (lungimea secvenței) și $d = 512$ (dimensiunea *embedding*-ului).

1. Determinați lungimea maximă a căii pentru ambele modele și comentați impactul acesteia asupra învățării dependențelor pe termen lung.
2. Comparați complexitatea computațională pentru ambele modele (în termeni de $O(\ldots)$). Care model ar fi mai lent pentru această secvență lungă?

Rezolvare P.1:

1. **Lungimea Maximă a Căii:**
   - **RNN:** $O(n) = O(1000)$. Informația trebuie să parcurgă 1000 de pași, îngreunând învățarea dependențelor pe distanțe lungi.
   - **Self-attention:** $O(1)$. Informația este transferată direct, făcând învățarea dependențelor pe termen lung mult mai ușoară.
2. **Complexitatea Computațională:**
   - **RNN:** $O(nd^2) = O(1000 \cdot 512^2) \approx O(262 \text{ miioane})$.
   - **Self-attention:** $O(n^2 d) = O(1000^2 \cdot 512) = O(1 \text{ miion} \cdot 512) \approx O(512 \text{ miiare})$.
   - **Concluzie:** Deși atenția proprie are un avantaj teoretic major în ceea ce privește lungimea căii, complexitatea sa pătratică ($O(n^2 d)$) o face **semnificativ mai lentă** (de aproximativ 2000 de ori) decât RNN-ul pentru secvențe foarte lungi ($n = 1000$).

Exercițiul P.2: Codificare Pozițională (Analiză Formula)

Se utilizează Codificarea Pozițională fixă (Ecuația 9). Se dorește injectarea informației poziționale pentru un *token* aflat la poziția $i = 20$ (rândul 20 al matricii P). Dimensiunea *embedding*-ului este $d = 128$.

Determinați ce formulă (sin sau cos) trebuie utilizată pentru elementele dimensionale $= 3$ și $= 4$ ale vectorului pozițional P. Scrieți expresiile complete pentru $p_{20,6}$ și $p_{20,7}$.

Rezolvare P.2:

- Formula folosește indici pari (2) pentru $\sin$ și impari ($2+1$) pentru os.
- Elementul dimensional $= 3$: Acesta corespunde coloanei $2 = 6$. Se folosește $\sin$.
- Elementul dimensional $= 4$: Acesta corespunde coloanei $2 = 8$ (nu este cerut).

1. **Expresia pentru $p_{20,6}$ (Coloana 6, $2 = 6 \quad = 3$):**

$$p_{20,6} = \sin \frac{20}{10000^{2 \cdot 3/128}} = \sin \frac{20}{10000^{6/128}} = \sin \frac{\mathbf{20}}{\mathbf{10000^{3/64}}}$$

2. **Expresia pentru $p_{20,7}$ (Coloana 7, $2 + 1 = 7 \quad = 3$):**

$$p_{20,7} = \text{os} \frac{20}{10000^{2 \cdot 3/128}} = \text{os} \frac{20}{10000^{6/128}} = \text{os} \frac{\mathbf{20}}{\mathbf{10000^{3/64}}}$$

Analiză: Coloanele 6 și 7 ale matricii P sunt complementare (sinus și cosinus) și folosesc aceeași frecvență, arătând decalajul menționat în sursă.

5.7

Aceasta este lecția definitorie a erei moderne în *deep learning* secvențial: **5.7 Transformer**. Propus inițial pentru sarcini de procesare a limbajului natural, modelul Transformer este remarcabil deoarece a reușit să înlocuiască complet straturile recurente (RNN) și convoluționale (CNN) cu mecanisme bazate exclusiv pe atenție, profitând de avantajele calculului paralel și a căilor de dependență scurte.

Ghid de Studiu Complet: 5.7 Transformer

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Modelul **Transformer**, deși a fost propus inițial pentru învățarea secvență-la-secvență pe date textuale, a devenit omniprezent într-o gamă largă de aplicații moderne de *deep learning*, inclusiv limbaj, viziune, vorbire și învățare prin consolidare. Spre deosebire de modelele anterioare bazate pe atenție care încă depindeau de RNN-uri pentru reprezentările de intrare, arhitectura Transformer se bazează **exclusiv pe mecanisme de atenție**, fără a folosi straturi convoluționale sau recurente. Datorită utilizării atenției proprii (*self-attention*), modelul beneficiază atât de calcul paralel, cât și de cea mai scurtă lungime maximă a căii ($O(1)$).

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectura:** Transformerul este o instanță a arhitecturii **Codificator-Decodificator**.
2. **Mecanismul de Bază:** Utilizează exclusiv **Atenția Proprie Multi-Cap (*Multi-head self-attention*)**, fără RNN sau CNN.
3. **Intrarea:** *Embedding*-urile secvențelor de intrare (sursă și țintă) sunt augmentate cu **Codificare Pozițională (*positional encoding*)**.
4. **Componente Cheie:** Fiecare sub-strat (în ambele părți) este înconjurat de o **Conexiune Reziduală** (inspirată de ResNet) urmată de **Normalizare pe Strat (*Layer Normalization*)**.
5. **Decodificarea:** Decodificatorul conține un al treilea sub-strat numit **Atenție Codificator-Decodificator (*encoder-decoder attention*)** și utilizează **Atenție Mascată (*masked attention*)** pentru a menține proprietatea autoregresivă.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

Arhitectura Generală (Figura 9)

- Arhitectura Transformer este compusă dintr-un **Codificator** și un **Decodificator**.
- Spre deosebire de Atenția Bahdanau (Figura 5), *embedding*-urile secvențelor de intrare (sursă) și de ieșire (țintă) sunt **adăugate cu codificare pozițională** înainte de a fi introduse în Codificator și Decodificator.
- Tanto Codificatorul cât și Decodificatorul sunt formate din stive de **straturi identice multiple**.

Codificatorul (Encoder)

- Codificatorul este o stivă de straturi identice, fiecare strat având **două sub-straturi** (sau sublayers).
  1. **Primul Sub-Strat:** Este un *multi-head self-attention pooling*.
  2. **Al Doilea Sub-Strat:** Este o rețea *position-wise feed-forward network* (FFN).
- În atenția proprie a Codificatorului, **queries, keys și values provin toate din ieșirile stratului Codificator anterior**.
- Ieșirea Codificatorului produce o reprezentare vectorială d-dimensională pentru fiecare poziție a secvenței de intrare.

Decodificatorul (Decoder)

- Decodificatorul este, de asemenea, o stivă de straturi identice cu conexiuni reziduale și normalizare pe strat.
- Decodificatorul inserează un **al treilea sub-strat** (Atenția Codificator-Decodificator) între cele două sub-straturi descrise în Codificator.

1. **Atenția Proprie Mascată (*Masked Self-Attention*):**
   - *Queries, keys și values* provin din ieșirile stratului Decodificator anterior.
   - Atenția este **mascată** pentru a păstra proprietatea **autoregresivă**.
   - Masca asigură că, la orice pas de decodificare, *query*-ul acordă atenție doar **pozițiilor din decodificator până la poziția curentă**.
   - Acest lucru garantează că predicția depinde doar de *token*-urile de ieșire care au fost deja generate.
2. **Atenția Codificator-Decodificator (*Encoder-Decoder Attention*):**
   - **Queries** provin din ieșirile stratului Decodificator anterior.
   - **Keys și Values** provin din **ieșirile Codificatorului Transformer** (întreaga secvență codificată a sursei).
3. **Position-wise FFN:** Similar cu Codificatorul.

Componentele Comune ("Add & Norm")

- **Conexiune Reziduală (*Residual Connection*):** Inspirată de designul ResNet (Secțiunea 3.10), o conexiune reziduală este utilizată în jurul ambelor/tuturor sub-straturilor.

    - Pentru ca adunarea reziduală $+\text{saer}()$ să fie fezabilă, sub-stratului i se cere să producă o ieșire cu aceeași dimensiune d ca și intrarea .
- **Normalizare pe Strat (*Layer Normalization*):** Adunarea reziduală este urmată imediat de normalizarea pe strat.
    - Normalizarea pe strat diferă de normalizarea pe *batch* prin faptul că normalizează **de-a lungul dimensiunii *feature*** (dimensiunea d), în timp ce normalizarea pe *batch* normalizează pe exemple în cadrul unui *mini-batch*.
    - Normalizarea pe strat este, în general, mai eficientă empiric în sarcinile NLP unde *input*-urile sunt adesea secvențe de lungime variabilă.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Position-wise Feed-Forward Network (FFN)

- **Explicație Simplificată (Procesare Locală, Global Partajată):** Deși FFN operează pe reprezentarea la **fiecare poziție** a secvenței independent (procesare locală), **aceleași ponderi** ale rețelei MLP sunt folosite pentru *toate* pozițiile secvenței. Este ca și cum ai avea același filtru de procesare aplicat fiecărui cuvânt, dar procesarea fiecărui cuvânt este independentă de celelalte.

Concept Dificil: Atenția Mascată (Masked Self-Attention)

- **Analogie Simplificată (Examen cu Răspunsuri Ascunse):** Când Decodificatorul generează o propoziție *token* cu *token* (proces autoregresiv), el trebuie să se bazeze doar pe ceea ce a tradus **până la momentul curent**.
- **Asta se întâmplă deoarece:** Dacă ar avea acces la *token*-urile viitoare (care sunt cunoscute doar în timpul antrenamentului, dar nu și în timpul predicției), ar "trișa". Mascarea atenției garantează că, la pasul t', *token*-ul t' poate vedea doar *token*-urile $1, 2, \ldots, t'$, dar nu și $t'+1, t'+2, \ldots$.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Secțiunea 5.6 (Atenția Proprie):** Transformerul exploatează avantajele Atenției Proprii: capacitatea de a parcurge lungimea secvenței în $O(1)$ pași, făcând învățarea dependențelor pe termen lung mult mai ușoară decât în RNN-uri. Nevoia de Codificare Pozițională derivă direct din această lipsă de recurență.
- **Conexiunea cu Secțiunea 5.4 (Bahdanau Attention):** Transformerul este o arhitectură Codificator-Decodificator (similar cu Bahdanau), dar înlocuiește RNN-urile Codificator/Decodificator cu module bazate pe atenție. În Decodificator, stratul **Atenție**

Codificator-Decodificator îndeplinește rolul de *cross-attention* (similar cu Bahdanau), unde *queries* vin din Decodificator și *keys/values* vin din Codificator.
- **Implicația Proiectării (Modularitate):** Utilizarea conexiunilor reziduale și a normalizării pe strat este esențială pentru a permite stivuirea (stacking) a multiplelor straturi identice, ceea ce contribuie la stabilitatea și performanța arhitecturii profunde.

---

5. EXERCITII (Calcul și Analiză)

Voi genera un exercițiu de calcul bazat pe numărul de parametri într-un strat de Codificator Transformer și un exercițiu conceptual axat pe componentele critice.

Exercițiul T.1: Calculul Parametrilor într-un Strat de Codificator (Bazat pe 5.5)

Calculați numărul total de parametri (fără bias-uri) pentru un **singur strat de Codificator** Transformer, care este compus din Atenție Proprie Multi-Cap și un FFN *position-wise*.

- Dimensiunea vectorului de *embedding* (Intrare/Ieșire Dimenisune d): $d = 100$.
- Numărul de Capete (Heads) h: $h = 4$.
- Dimensiunea de Proiecție per Cap p_q, p_k, p_v: $p_q = p_k = p_v = 25$. (Astfel $h \cdot p_v = 100 = d$).
- Dimensiunea Internă FFN d_{ff}: $d_{ff} = 4d = 400$.

Componente de calculat:

1. Parametrii pentru Atenția Proprie Multi-Cap (Proiecțiile Q, K, V și Ieșirea $\mathbf{W}_o$).
2. Parametrii pentru FFN (Două straturi liniare: $_1$ și $_2$).
3. Total Parametri pe Strat.

Rezolvare T.1:

1. **Atenția Proprie Multi-Cap (Self-Attention):**
   - **Proiecțiile (Q, K, V):** Deoarece $h \cdot p_d = d$, se poate folosi o singură matrice mare ($d \times d$) care este împărțită logic (sau h matrici mai mici).
   - $^{(q)}, ^{(k)}, ^{(v)} \in \mathbb{R}^{d \times d}$. Total 3 matrici $d \times d$.

$$\text{ParametriProiecții} = 3 \times (d \times d) = 3 \times (100 \times 100) = 30,000$$

   - **Matricea de Ieșire ($\mathbf{W}_o$):** Ieșirea concatenată este $h \cdot p_v = 100$. Ieșirea finală este $d = 100$.

$$\text{Parametri}_o = d \times d = 100 \times 100 = 10,000$$

$$\text{otatențieProprie} = 30,000 + 10,000 = \mathbf{40,000}$$

2. **Position-wise Feed-Forward Network (FFN):**
   - $_1(d \quad d_{ff})$**:** $100 \times 400 = 40,000$

- $_2(d_{ff} \quad d)$**:** $400 \times 100 = 40,000$

$$\text{ota} = 40,000 + 40,000 = \mathbf{80,000}$$

3. **Total Parametri pe Stratul de Codificator:**

$$\text{ota} = 40,000(\text{tenție}) + 80,000() = \mathbf{120,000}$$

Exercițiul T.2: Rolul Normalizării pe Strat (Conceptual)

De ce preferă arhitectura Transformer Normalizarea pe Strat (*Layer Normalization*) în locul Normalizării pe Batch (*Batch Normalization*), mai ales în sarcinile de procesare a limbajului natural (NLP)?

Rezolvare T.2:

1. **Diferența de Normalizare:** Normalizarea pe *Batch* normalizează *feature*-urile peste exemple în *mini-batch*. Normalizarea pe Strat normalizează **de-a lungul dimensiunii *feature*** (vectorul d-dimensional).
2. **Secvențe de Lungime Variabilă:** Input-urile în NLP sunt adesea secvențe de lungime variabilă.
3. **Stabilitate:** Normalizarea pe Strat s-a dovedit a fi **empiric mai eficientă** decât normalizarea pe *batch* în sarcinile NLP, oferind o mai mare stabilitate pentru antrenarea rețelelor profunde cu secvențe de lungime variabilă.

5.8

Absolut! Având în vedere solicitarea de a genera exerciții de examen, în special cele de calcul care aplică formulele mecanismelor de atenție, și bazându-mă pe modelul complex oferit în surse (Exercițiul 5), am pregătit un set variat de probleme.

Aceste exerciții acoperă:

1. Calculul Atenției Aditive (tipic Bahdanau).
2. Calculul Atenției Produsului Scalar Scalat (tipic Transformer/Self-Attention).
3. Analiza conceptuală și dimensională a componentelor cheie.

Exerciții de Examen: Mecanisme de Atenție (Secțiunea 5)

Secțiunea I: Calculul Atenției (Additive & Scaled Dot-Product)

Exercițiul A.1: Calculul Atenției Aditive (Similar Exercițiului 5 din Sursă)

Definiți un mecanism de Atenție Aditivă (similar Bahdanau Attention). Avem $m = 2$ perechi *key-value*. $k = 2$ (dimensiunea Key), $v = 2$ (dimensiunea Value), $q = 2$ (dimensiunea Query). Dimensiunea stratului ascuns este $h = 2$.

- **Query:** $\mathbf{q} = [1\ 1]$
- **Perechi Key-Value:**

$$(\mathbf{k}_1, \mathbf{v}_1) = ([0\ 1], [3\ 1]), \quad (\mathbf{k}_2, \mathbf{v}_2) = ([1\ 0], [1\ 2])$$

- **Ponderi (folosim aceleași dimensiuni $h = 2, q = 2, k = 2$):**

$$\mathbf{w}_v^\top = [1 \quad 0] \in \mathbb{R}^{1\times 2}, \quad \mathbf{W}_q = [1 \quad 1\ 0 \quad 0] \in \mathbb{R}^{2\times 2}, \quad \mathbf{W}_k = [0 \quad 1\ 1 \quad 0] \in \mathbb{R}^{2\times 2}$$

Formule esențiale:

$$a(\mathbf{q}, \mathbf{k}) = \mathbf{w}_v^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}) \in \mathbb{R}$$

$$\alpha(\mathbf{q}, \mathbf{k}) = \mathrm{softmax}(a(\mathbf{q}, \mathbf{k}))$$

$$f(\mathbf{q}, \ldots) = \sum_{i=1}^{m} \alpha(\mathbf{q}, \mathbf{k}_i)\mathbf{v}_i$$

Presupuneri necesare: $\tanh(1) \approx 0.76$, $\tanh(2) \approx 0.96$, $\exp(0.76) \approx 2.14$, $\exp(0.96) \approx 2.61$.

1. Calculați scorul de aliniere $a(\mathbf{q}, \mathbf{k}_1)$.
2. Calculați scorul de aliniere $a(\mathbf{q}, \mathbf{k}_2)$.
3. Calculați ponderile de atenție α_1 și α_2.
4. Calculați output-ul final $f(\mathbf{q}, \ldots)$.

Rezolvare A.1:

1. Calculul $a(\mathbf{q}, \mathbf{k}_1)$:

$$\mathbf{W}_q\mathbf{q} = [1 \quad 1\ 0 \quad 0]\,[1\ 1] = [2\ 0]$$

$$\mathbf{W}_k\mathbf{k}_1 = [0 \quad 1\ 1 \quad 0]\,[0\ 1] = [1\ 0]$$

$$\text{Intrare } \tanh = [2\ 0] + [1\ 0] = [3\ 0]$$

$$a(\mathbf{q}, \mathbf{k}_1) = [1 \quad 0]\tanh[3\ 0] \approx (1 \cdot \tanh(3)) + (0 \cdot \tanh(0)) \approx \tanh\mathbf{(3)} \approx \mathbf{0.995}$$

2. Calculul $a(\mathbf{q}, \mathbf{k}_2)$:

$$\mathbf{W}_q\mathbf{q} = [2\ 0]$$

$$\mathbf{W}_k\mathbf{k}_2 = [0 \quad 1\ 1 \quad 0]\,[1\ 0] = [0\ 1]$$

$$\text{Intrare } \tanh = [2\ 0] + [0\ 1] = [2\ 1]$$

$$a(\mathbf{q}, \mathbf{k}_2) = [1 \quad 0]\tanh[2\ 1] \approx (1 \cdot \tanh(2)) + (0 \cdot \tanh(1)) \approx \tanh\mathbf{(2)} \approx \mathbf{0.96}$$

3. Calculul Ponderilor de Atenție (α):

- $a_1 = 0.995, a_2 = 0.96.$
- $\exp(a_1) \approx 2.70$, $\exp(a_2) \approx 2.61.$
- Denominator $D = 2.70 + 2.61 = 5.31.$

$$\alpha_1 = \frac{2.70}{5.31} \approx \mathbf{0.508}$$

$$\alpha_2 = \frac{2.61}{5.31} \approx \mathbf{0.492}$$

4. Calculul Output-ului Final ($f(\mathbf{q}, \ldots)$):

$$f(\mathbf{q}, \ldots) = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2$$

$$f(\mathbf{q}, \ldots) \approx 0.508\,[3\ 1] + 0.492\,[1\ 2]$$

$$f(\mathbf{q}, \ldots) \approx [1.524\ 0.508] + [0.492\ 0.984] = [2.016\ 1.492]$$

Exercițiul A.2: Calculul Atenției Produsului Scalar Scalat (Self-Attention)

Utilizați aceleași *Queries, Keys* și *Values* ca în Exercițiul A.1, dar folosiți formula pentru Atenția Produsului Scalar Scalat.

$$\mathbf{q} = [1\ 1], \quad \mathbf{k}_1 = [0\ 1], \quad \mathbf{k}_2 = [1\ 0]$$

$$v_1 = [3\ 1], \quad v_2 = [1\ 2]$$

- $d = k = 2$. Factorul de scalare: $1/\sqrt{2} \approx 0.707$.
- Presupuneri necesare: $\exp(0.707) \approx 2.028, \exp(0) = 1.0.$

1. Calculați scorurile de aliniere $a(\mathbf{q}, \mathbf{k}_1)$ și $a(\mathbf{q}, \mathbf{k}_2)$.
2. Calculați ponderile de atenție α_1 și α_2.
3. Calculați output-ul final $f(\mathbf{q}, \ldots)$.

Rezolvare A.2:

1. **Calculul Scorurilor de Aliniere ($a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}/\sqrt{d}$):**
   - $a_1 = ([1 \quad 1]\,[0\ 1]) \cdot 0.707 = (1) \cdot 0.707 = \mathbf{0.707}$
   - $a_2 = ([1 \quad 1]\,[1\ 0]) \cdot 0.707 = (1) \cdot 0.707 = \mathbf{0.707}$
2. **Calculul Ponderilor de Atenție (α):**
   - Deoarece ambele scoruri sunt identice (0.707), ponderile vor fi egale.
   - Denominator $D = \exp(0.707) + \exp(0.707) \approx 2.028 + 2.028 = 4.056.$
   - $$\alpha_1 = \frac{2.028}{4.056} = \mathbf{0.5}$$
   - $$\alpha_2 = \mathbf{0.5}$$

3. **Calculul Output-ului Final ($f(\mathbf{q}, \ldots)$):**

$$f(\mathbf{q}, \ldots) = 0.5\mathbf{v}_1 + 0.5\mathbf{v}_2$$

$$f(\mathbf{q}, \ldots) = 0.5\,[3\ 1] + 0.5\,[1\ 2] = [1.5\ 0.5] + [0.5\ 1.0] = [2.0\ 1.5]$$

Secțiunea II: Exerciții de Teorie și Analiză Dimensională

Exercițiul T.1: Analiză Dimensională (Multi-head attention)

Un modul de Atenție Multi-Cap (*Multi-head attention*) într-un Transformer folosește $h = 8$ capete. Dimensiunea vectorului de intrare/ieșire este $d = 256$.

1. Determinați dimensiunea de proiecție a *Values* per cap (p_v) dacă se presupune că $h \cdot p_v = d$.
2. Calculați numărul total de parametri necesari doar pentru proiecțiile liniare ale *Query, Key* și *Value* ($\mathbf{W}^{(q)}, \mathbf{W}^{(k)}, \mathbf{W}^{(v)}$) pentru **toate** cele 8 capete, presupunând că sunt proiectate folosind matrici distincte.

Rezolvare T.1:

1. **Dimensiunea de Proiecție per Cap (p_v):**

$$p_v = d/h = 256/8 = \mathbf{32}$$

2. **Total Parametri Proiecții:**
   - Fiecare cap proiectează Q, K și V de la dimensiunea $d = 256$ la dimensiunea $p = 32$.
   - Parametri per matrice: $d \times p = 256 \times 32 = 8192$.
   - Parametri per cap (3 matrici): $3 \times 8192 = 24576$.
   - Total pentru $h = 8$ capete: $8 \times 24576 = \mathbf{196608}$ parametri.

Exercițiul T.2: Atenția Proprie și Ordinea (Conceptual)

Explicați de ce un strat de **Atenție Proprie** (Self-Attention) nu poate funcționa corect fără **Codificare Poziațională** într-o arhitectură Transformer.

Rezolvare T.2:

1. **Lipsa Ordinii:** Atenția proprie înlocuiește operațiile recurente ($O(n)$ operații secvențiale) cu operații paralele ($O(1)$ operații secvențiale) [conversation history, Secțiunea 5.6].
2. **Informația Pierdută:** Deoarece calculul se face în paralel pe baza similarității dintre vectorii *Query* și *Key* (indiferent de poziția lor), modelul **pierde informația despre ordinea** absolută sau relativă a *token*-urilor în secvență [conversation history, Secțiunea 5.6].
3. **Soluția:** Codificarea Poziațională este injectată în *embedding*-ul de intrare pentru a furniza modelului aceste informații de ordine, care sunt cruciale pentru înțelegerea sintaxei și semanticii secvenței.

Exercițiul T.3: Diferența dintre Bahdanau și Scaled Dot-Product (Teoretic)

Specificați principala diferență arhitecturală și motivația în alegerea funcției de scorare între **Atenția Aditivă (tipic Bahdanau)** și **Atenția Produsului Scalar Scalat** în contextul Q-K-V.

Rezolvare T.3:

1. **Atenția Aditivă (Bahdanau):**
   - **Funcție de Scorare:** Utilizează o transformare neliniară (tanh) și parametri învățabili ($\mathbf{W}_q, \mathbf{W}_k, \mathbf{w}_v$).
   - **Motivație/Avantaj:** Este preferată atunci când dimensiunea *Query* și dimensiunea *Key* sunt **diferite** ($q \neq k$), cum ar fi în cazul Bahdanau unde starea decodificatorului și stările codificatorului pot avea dimensiuni diferite [conversation history, Secțiunea 5.3].
2. **Atenția Produsului Scalar Scalat:**
   - **Funcție de Scorare:** Utilizează un produs scalar simplu, împărțit la $\sqrt{d}$ pentru stabilizare.
   - **Motivație/Avantaj:** Este mult mai **eficientă computațional** și este utilizată exclusiv când dimensiunile *Query* și *Key* sunt **identice** ($q = k = d$) [conversation history, Secțiunea 5.3]. Scalarea cu $\sqrt{d}$ este necesară pentru a menține varianța constantă și a asigura stabilitatea Softmax în timpul antrenamentului.

Absolut! Voi genera un set nou de exerciții, incluzând probleme de calcul bazate pe formulele din sursa furnizată (Exercițiul 5) și probleme conceptuale legate de funcționarea mecanismelor de atenție.

Set Suplimentar de Exerciții de Examen

Secțiunea I: Calculul Atenției Bazate pe Sursa (Atenția Produsului Scalar Scalat)

Vom folosi ca model calculul Atenției Produsului Scalar Scalat din sursă.

Exercițiul C.1: Calculul Atenției Produsului Scalar Scalat

Se folosesc setul de date și parametri din Sursa (Exercițiul 5), unde $m = 3$ perechi *key-value*, $k = 3, v = 2, q = 3$.

- **Query:** $\mathbf{q} = [1\ 2\ 3]^\top$
- **Keys & Values:**

$$(\mathbf{k}_1, \mathbf{v}_1) = ([2\ 0\ 1], [-1\ 1]), \quad (\mathbf{k}_2, \mathbf{v}_2) = ([0\ \ -1\ 0], [-1\ 0]), \quad (\mathbf{k}_3, \mathbf{v}_3) = ([0\ 2\ 0], [0\ 1])$$

Formula de Scorare: $a(\mathbf{q}, \mathbf{k}) = \mathbf{q}^\top \mathbf{k}/\sqrt{d}$, unde $d = 3$. $1/\sqrt{3} \approx 0.577$.

Presupuneri necesare: $e^{2.885} \approx 17.9, e^{1.154} \approx 3.17, e^{2.308} \approx 10.05$. (Notă: Acestea sunt aproximările pentru scorurile calculate în sursă).

1. Verificați scorurile de aliniere $a(\mathbf{q}, \mathbf{k}_1)$, $a(\mathbf{q}, \mathbf{k}_2)$ și $a(\mathbf{q}, \mathbf{k}_3)$.
2. Calculați ponderile de atenție $\alpha_1, \alpha_2, \alpha_3$ (folosind aproximările exponențiale date).
3. Calculați output-ul final $f(\mathbf{q}, \ldots)$.

Rezolvare C.1:

1. **Verificarea Scorurilor de Aliniere ($a(\mathbf{q}, \mathbf{k})$):**
   - Sursa calculează scorurile: $a(\mathbf{q}, \mathbf{k}_1) = 5/\sqrt{3} \approx 5 \cdot 0.577 = 2.885$.
   - $a(\mathbf{q}, \mathbf{k}_2) = 2/\sqrt{3} \approx 2 \cdot 0.577 = 1.154$.
   - $a(\mathbf{q}, \mathbf{k}_3) = 4/\sqrt{3} \approx 4 \cdot 0.577 = 2.308$. *(Scorurile sunt corecte conform formulei din sursă)*.
2. **Calculul Ponderilor de Atenție (α):**
   - Termeni exponențiali: $e^{2.885} \approx 17.9, e^{1.154} \approx 3.17, e^{2.308} \approx 10.05$.
   - Denominator $D = 17.9 + 3.17 + 10.05 = 31.12$.
   - $\alpha_1 = 17.9/31.12 \approx \mathbf{0.575}$
   - $\alpha_2 = 3.17/31.12 \approx \mathbf{0.102}$
   - $\alpha_3 = 10.05/31.12 \approx \mathbf{0.323}$ *(Suma ponderilor* $\approx 0.575 + 0.102 + 0.323 = 1.000$*)*.
3. **Calculul Output-ului Final ($f(\mathbf{q}, \ldots)$):**

$$f(\mathbf{q}, \ldots) = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3$$

$$f(\mathbf{q}, \ldots) \approx 0.575\,[-1\ 1] + 0.102\,[-1\ 0] + 0.323\,[0\ 1]$$

$$f(\mathbf{q}, \ldots) \approx [-0.575\ 0.575] + [-0.102\ 0] + [0\ 0.323]$$

$$f(\mathbf{q}, \ldots) = [-0.677\ 0.898]$$

Secțiunea II: Exerciții Conceptual-Analitice (Bahdanau & Transformer)

Exercițiul T.4: Rolul Elementelor de Atenție (Attention Cues)

Explicați corespondența dintre cele două elemente fundamentale (cues) ale atenției umane și componentele Q-K-V din mecanismul computațional.

Rezolvare T.4:

Mecanismul de atenție computațional este inspirat de cadrul de atenție cu două componente [conversation history, Secțiunea 5.1]:

1. **Elementul Volitional (*Volitional Cue*):** Acesta este controlul conștient, dependent de sarcină, care determină ce informație este căutată. În mecanismele de atenție, acest rol

este îndeplinit de **Query ($\mathbf{q}$)**. Exemplu: În Bahdanau Attention, $\mathbf{q}$ este starea decodificatorului, care caută următorul cuvânt [conversation history, Secțiunea 5.4].

2. **Elementul Nonvolitional (*Nonvolitional Cue* / Saliency):** Acesta este proeminența sau proprietățile intrinseci ale obiectelor din mediu [conversation history, Secțiunea 5.1]. Acest rol este îndeplinit de **Key ($\mathbf{k}$)**, care permite *query*-ului să evalueze relevanța *value*-ului [conversation history, Secțiunea 5.1].
3. **Value ($\mathbf{v}$):** Acesta reprezintă **Input-ul Senzorial** (datele brute sau reprezentările acestora) care este filtrat și utilizat pentru a forma ieșirea [conversation history, Secțiunea 5.1].

Exercițiul T.5: Multi-head attention (Conceptual)

În contextul Transformer, de ce este benefic să se utilizeze **Atenția Multi-Cap** ($h \ 1$), mai degrabă decât o singură Atenție Proprie ($h = 1$) cu aceeași dimensiune totală de ieșire d?

Rezolvare T.5:

Atenția Multi-Cap (MHA) este benefică deoarece permite modelului să combine cunoștințe provenite din **comportamente diferite** ale mecanismului de atenție [conversation history, Secțiunea 5.5].

1. **Subspații de Reprezentare Diferite:** Fiecare cap utilizează **proiecții liniare învățate independent** [conversation history, Secțiunea 5.5]. Aceste proiecții transformă Q, K și V în subspații diferite, permițând fiecărui cap să învețe să acorde atenție la *feature*-uri sau relații distincte (ex: un cap se poate concentra pe sintaxă, altul pe referințe de lungă durată) [conversation history, Secțiunea 5.5].
2. **Diversitate:** Dacă s-ar folosi un singur cap mare, acesta ar avea o singură perspectivă asupra relațiilor din secvență. MHA asigură că modelul captează **o gamă mai largă de dependențe** (de exemplu, atât dependențe pe termen scurt, cât și pe termen lung).

Exercițiul T.6: Rolul Scalării în Produsul Scalar Scalat (Teoretic)

Explicați de ce formula Atenției Produsului Scalar Scalat necesită împărțirea la $\sqrt{d}$.

Rezolvare T.6:

1. **Creșterea Varianței:** Dacă vectorii *Query* ($\mathbf{q}$) și *Key* ($\mathbf{k}$) sunt vectori aleatori independenți cu medie zero și varianță unitară, produsul scalar $\mathbf{q}^\top \mathbf{k}$ are o varianță care crește proporțional cu dimensiunea d a vectorilor [conversation history, Secțiunea 5.3].
2. **Problema Softmax:** Când varianța (și, implicit, mărimea absolută a scorurilor) este mare, intrarea în funcția Softmax devine foarte mare. Softmax-ul aplicat unor valori mari produce o distribuție foarte polarizată (unde o pondere este aproape 1, iar restul sunt aproape 0) [conversation history, Secțiunea 5.3].
3. **Instabilitatea Gradientului:** O distribuție polarizată duce la **gradienți extrem de mici** (gradientul Softmax devine mic în regiunile saturate), încetinind sau stabilizând în mod

necorespunzător antrenamentul.

4. **Soluția: Împărțirea la $\sqrt{d}$** normalizează varianța produsului scalar, menținând-o constantă (aproximativ egală cu 1), ceea ce stabilizează Softmax-ul și asigură un antrenament mai robust [conversation history, Secțiunea 5.3].